

Organization

About myself

Philipp Schuster

BSc and MSc in Koblenz in Computer Visualistics.

PhD in Tübingen in Programming Languages.

PostDoc in Tübingen in Software Engineering.

Teaching Specialist for Practical Computer Science.

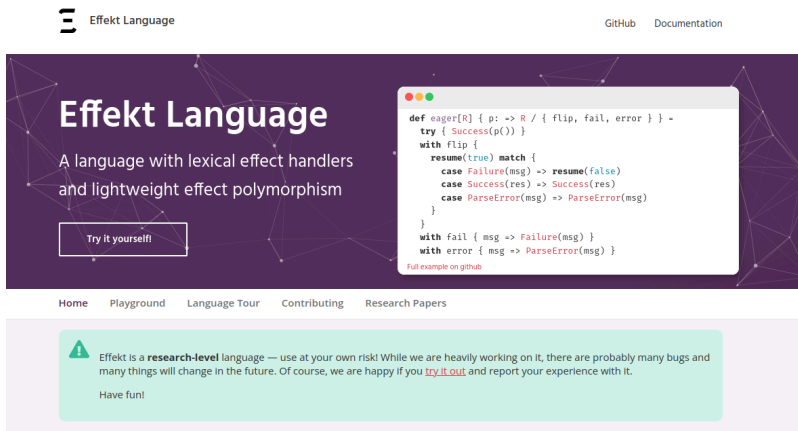
Started in May, this is my first lecture.

Research Interest

All programming languages.

Compilers for languages with non-sequential control flow.

Backend developer on [Effekt](#).



The screenshot shows the website for the Effekt Language. At the top left is the logo "Effekt Language" with a hamburger menu icon. To the right are links for "GitHub" and "Documentation". The main header area has a dark purple background with a network diagram. It features the text "Effekt Language" and "A language with lexical effect handlers and lightweight effect polymorphism". Below this is a button that says "Try it yourself!". To the right of the text is a code editor window showing a snippet of Effekt code. At the bottom of the page is a light green banner with an information icon and a disclaimer: "Effekt is a research-level language — use at your own risk! While we are heavily working on it, there are probably many bugs and many things will change in the future. Of course, we are happy if you [try it out](#) and report your experience with it. Have fun!".

Effekt Language

GitHub Documentation

Effekt Language

A language with lexical effect handlers and lightweight effect polymorphism

Try it yourself!

```
def eager[R] { p: => R / { flip, fail, error } } =  
  try { Success(p()) }  
  with flip {  
    resume(true) match {  
      case Failure(msg) => resume(false)  
      case Success(res) => Success(res)  
      case ParseError(msg) => ParseError(msg)  
    }  
  }  
  with fail { msg => Failure(msg) }  
  with error { msg => ParseError(msg) }
```

Full example on [github](#)

Home Playground Language Tour Contributing Research Papers

! Effekt is a **research-level** language — use at your own risk! While we are heavily working on it, there are probably many bugs and many things will change in the future. Of course, we are happy if you [try it out](#) and report your experience with it. Have fun!

Lecture

Two lectures per week: Tuesday 14:00 and Friday 14:00.

One theoretical topic per week.

One practical technology per week.

Mix of slides and live coding.

No script, do take notes!

Lab

One lab per week: Tuesday 12:00.

Practice last week's topic and technology.

Same exercise sheet for homework and lab.

Homework is not checked.

Use ChatGPT, Claude, etc. to prepare for lab.

Everyone has to have live coded in lab twice to be admitted to the exam.

Forum

<https://ps-forum.cs.uni-tuebingen.de/invites/7oBoGCSEBF>

Our central communication platform. No Emails!

Distribution of exercise sheets.

Questions about lecture content and administration.

Exam

Code reading and code writing tasks.

Multiple choice questions to test your knowledge.

Main exam: 10.02.25, 14:00

Make-up exam: 24.03.25, 14:00

This course

Parallel, concurrent, and distributed *programming*.

Hands-on course in practical computer science.

Focus on what works, not on what doesn't.

Advanced programming in multiple languages.

Broad overview over many concepts.

Parallel, Concurrent, Distributed

They are different!

Parallelism

Same result but faster!

Examples:

- Assembly lines
- Audio encoding
- Graphics rendering
- Neural network inference

Must involve multiple processors.

Concurrency

Events with unknowable order!

Examples:

- Traffic
- Central database
- Graphical user interface
- Web server

May involve only a single processor.

Distribution

Running in different locations!

Examples for distributed parallelism:

- Search engines
- Neural network training
- Weather simulation

Examples for distributed concurrency:

- Chat software
- Multiplayer games
- Collaborative editing

Unreliable communication.

Throughput versus Latency

Throughput: items per time.

Latency: time per item.

The two might be in conflict.

Parallelism: often we want to optimize throughput.

Concurrency: often we want to optimize latency.

Distribution: often we need to balance both.

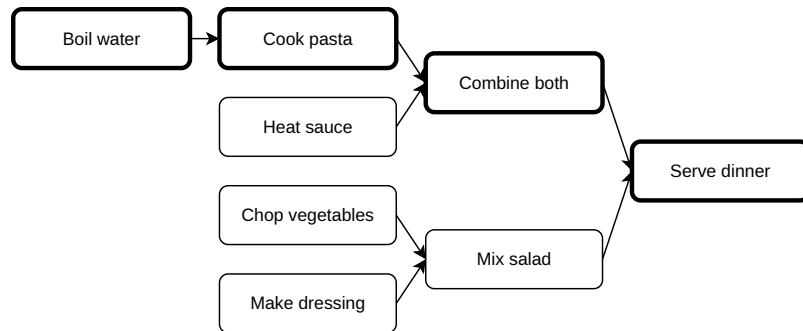
Work versus Span

Work: total duration of operations.

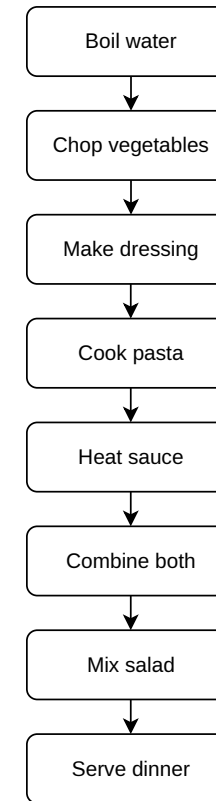
Span: duration of critical path.

Speedup: Work divided by Span.

Parallel:



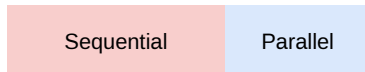
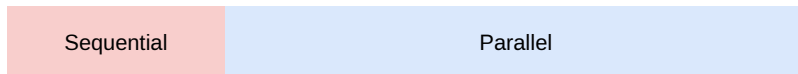
Sequential:



Amdahl's Law

[...] the effort expended on achieving high parallel processing rates is wasted unless it is accompanied by achievements in sequential processing rates of very nearly the same magnitude.

Gene M. Amdahl. 1967. Validity of the single processor approach to achieving large scale computing capabilities.



$$\text{Speedup} = \text{Work} / (\text{Sequential} + \text{Parallel} / \text{Processors})$$

Gustafson's Law

[...] in practice, the problem size scales with the number of processors.

John L. Gustafson. 1988. Reevaluating Amdahl's law.

Examples: image resolution, weights in neural networks, number of websites.

Tools

Nix

Reproducible development environments.

<https://nix.dev/install-nix>

Enable nix-command and flakes.

In file `~/.config/nix/nix.conf` :

```
experimental-features = nix-command flakes
```

demo/flake.nix

Benchmarking

Statistics: report empirical mean and standard deviation over multiple runs.

Reproducibility: pin down hardware, operating system, compilers, libraries, ...

Warmups: beware of power saving, thermal throttling, memory and file caching, just-in-time compilation, ...

Baseline: compare against the fastest alternative.

Scaling: compare on different input sizes.

Benchmarking Tools

Chronos

For quick iteration.

```
chronos './main1' './main2' './main3'
```

Hyperfine

For reportable results.

```
hyperfine -w 10 --export-csv results.csv './main1' './main2' './main3'
```

demo/main1.c

Summary and Outlook

Parallelism and Concurrency are different.

<https://inside.java/2021/11/30/on-parallelism-and-concurrency/>

Distribution can be about both.

Do not guess performance, measure it.

Next week: regular data parallelism.

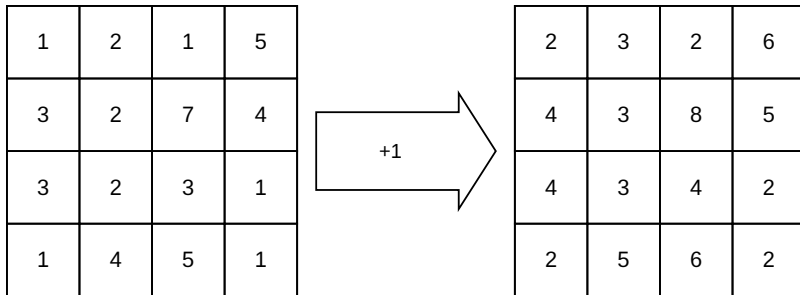
Regular Data Parallelism

Regular Arrays

Multi-dimensional flat arrays (not ragged).

Examples: raster graphic pictures, medical imaging volumes, neural network weights, ..., vectors, matrices, tensors, ...

Single Instruction Multiple Data (SIMD) parallelism.



When to use it?

Whenever applicable! This is what works!

This is the simplest yet an immensely important form of parallelism!

"Embarrassingly Parallel"

Examples: physics simulation, scientific computation, machine learning, ...

"Number Crunching"

PyTorch

<https://pytorch.org/>

[...] and does so while maintaining performance comparable to the fastest current libraries for deep learning. This combination has turned out to be very popular in the research community with, for instance, 296 ICLR 2019 submissions mentioning PyTorch.

Adam Paszke, Sam Gross, Francisco Massa, et al. 2019. PyTorch: an imperative style, high-performance deep learning library.

Python

Interpreted, untyped programming language.

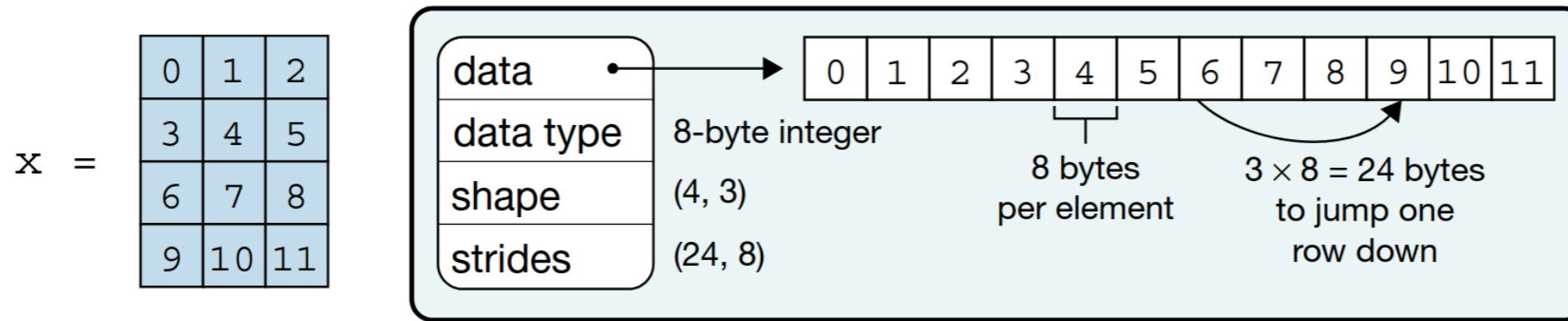
```
factorial = 1  
  
for i in range(2, n + 1):  
    factorial *= i  
  
print(factorial)
```

Created by Guido van Rossum in 1991.

Python Package Index: rich ecosystem of useful packages.

torch.Tensor

<https://docs.pytorch.org/docs/stable/tensors.html>



Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. Array programming with NumPy.

Element-wise Operations

Assuming `a` and `b` are vectors, this exploits parallelism:

```
c = a + b
```

It is that simple!

demo/basics.py

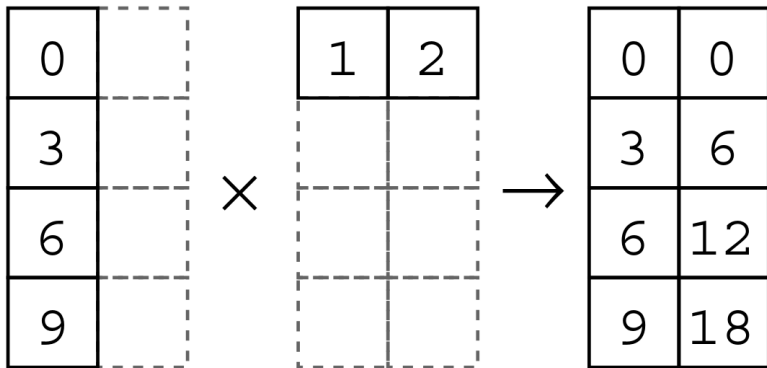
Broadcasting

What does the following mean when `a` is a vector?

```
2 * a
```

Scalar multiplication!

Not limited to scalars:



Broadcasting in PyTorch

<https://docs.pytorch.org/docs/stable/notes/broadcasting.html>

Two tensors are “broadcastable” if the following rules hold:

- Each tensor has at least one dimension.
- When iterating over the dimension sizes, starting at the trailing dimension, the dimension sizes must either be equal, one of them is 1, or one of them does not exist.

If two tensors x , y are “broadcastable”, the resulting tensor size is calculated as follows:

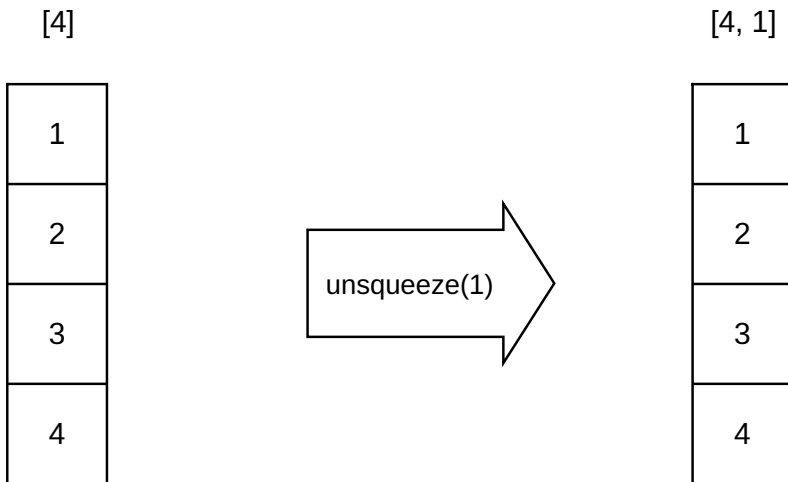
- If the number of dimensions of x and y are not equal, prepend 1 to the dimensions of the tensor with fewer dimensions to make them equal length.
- Then, for each dimension size, the resulting dimension size is the max of the sizes of x and y along that dimension.

torch.unsqueeze

```
torch.unsqueeze(input, dim) -> Tensor
```

Returns a new tensor with a dimension of size one inserted at the specified position.

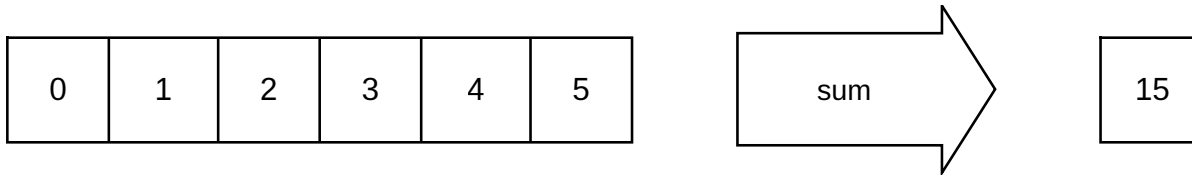
Data stays the same, shape is extended.



demo/broadcasting.py

Reduction

Aggregation functions `sum`, `max`, etc.



Associative operation: $(a + b) + c = a + (b + c)$

$((((0 + 1) + 2) + 3) + \dots)$

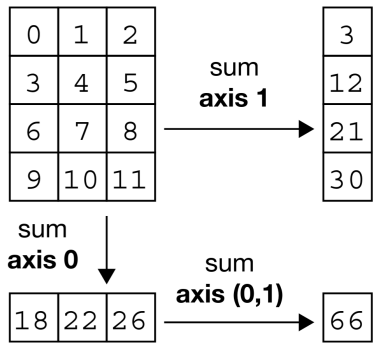
Parallel tree-shaped reduction:

$(0 + 1) + (2 + 3) + \dots$

Reduction in PyTorch

```
x = torch.tensor([[0, 1, 2], [3, 4, 5], [6, 7, 8], [9, 10, 11]])  
  
print(x.sum(dim=1))  
print(x.sum(dim=0))  
print(x.sum(dim=(0, 1)))
```

Parallel along the other axis:



Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. Array programming with NumPy.

demo/reductions.py

Example: Softmax

https://en.wikipedia.org/wiki/Softmax_function

$$\sigma(\mathbf{z})_i = \frac{e^{\beta(z_i - m)}}{\sum_{j=1}^K e^{\beta(z_j - m)}}$$

where $m = \max_i z_i$ is the largest factor involved.

```
def softmax(x: torch.Tensor) -> torch.Tensor:  
    y = torch.exp(x - x.max())  
    return y / y.sum()
```

demo/softmax.py

Vectorization

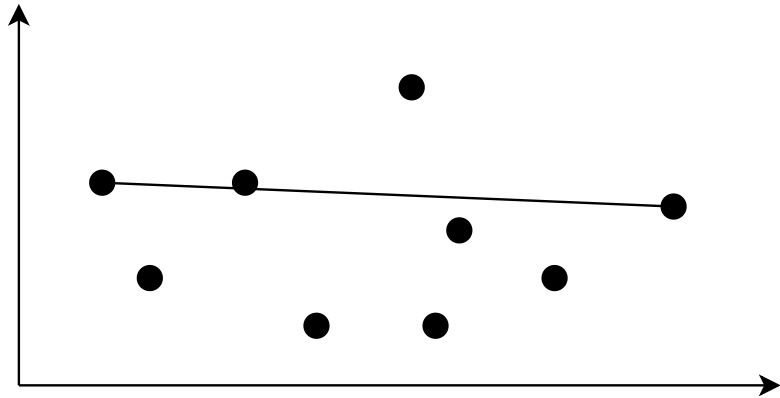
Sequential loops are slow, parallel loops are fast.

Rewrite programs to use tensor operations instead of loops!

Beware of data dependencies between loop iterations.

Vectorization Example

Maximum distance between points in 2D space.



```
for i in range(size):  
    for j in range(size):  
        distances[i, j] = torch.norm(points[i] - points[j])  
  
print(distances.max())
```

demo/distances.py

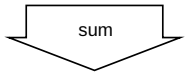
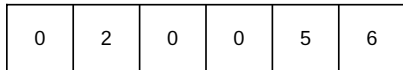
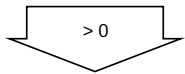
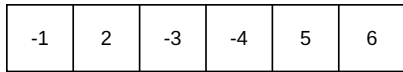
Conditions and Masking

There is no control flow in data parallelism.

We must emulate control flow with conditional data flow.

This might perform unnecessary work.

Example: sum of positive numbers.



Conditions and Masking in PyTorch

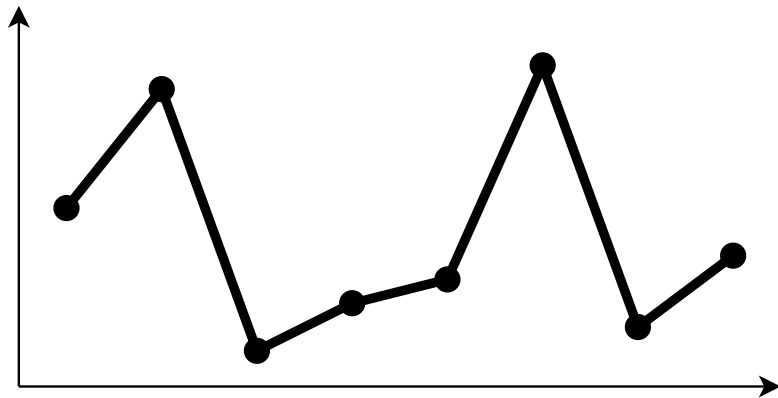
```
x = torch.tensor([-1, 2, -3, -4, 5, 6])  
mask = x > 0  
filtered = torch.where(mask, x, torch.tensor(0))  
result = filtered.sum()
```

demo/conditions.py

Convolution

Locality between indexes in domain.

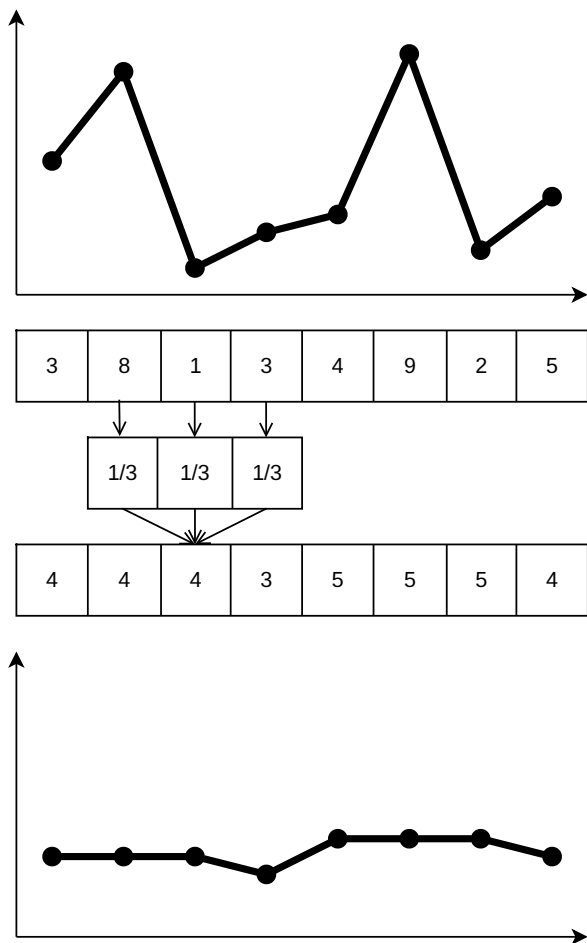
Examples: Sound and Image Processing, Tomograms, Fluid simulations, ...



3	8	1	3	4	9	2	5
---	---	---	---	---	---	---	---



Convolution in PyTorch



Sequential

```
input = [3, 8, 1, 3, 4, 9, 2, 5]
result = [0] * len(input)

for i in range(1, len(input) - 1):
    result[i] = (1/3) * input[i-1] + (1/3) * input[i] + (1/3) * input[i+1]
```

Parallel

```
input = torch.tensor([3, 8, 1, 3, 4, 9, 2, 5])
kernel = torch.tensor([1/3, 1/3, 1/3])

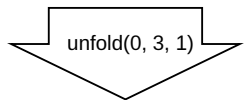
windows = input.unfold(0, 3, 1)
result = (kernel * windows).sum(dim=1)
```


torch.unfold

```
Tensor.unfold(dimension, size, step) -> Tensor
```

Returns a view of the original tensor which contains all slices of size `size` from `self` tensor in the `dimension` dimension. Step between two slices is given by `step`.

3	8	1	3	4	9	2	5
---	---	---	---	---	---	---	---

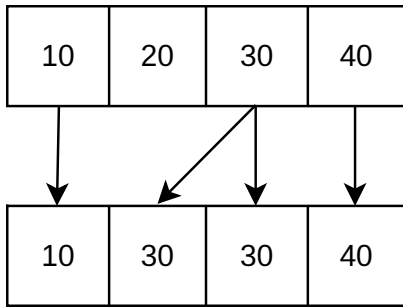


3	8	1	3	4	9
8	1	3	4	9	2
1	3	4	9	2	5

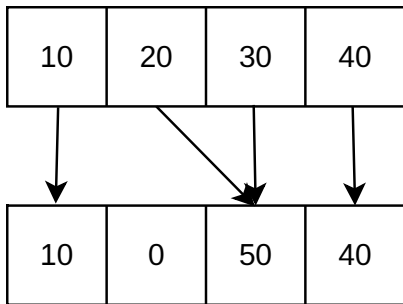
demo/convolution.py

Irregular data parallelism

Gather



Scatter



Irregular data parallelism in PyTorch

Gather

```
x = torch.tensor([10, 20, 30, 40])
i = torch.tensor([0, 2, 2, 3])
r = x[i] # tensor([10, 30, 30, 40])
```

Scatter

```
r = torch.zeros(4)
x = torch.tensor([10, 20, 30, 40])
i = torch.tensor([0, 2, 2, 3])
r = r.scatter_add(0, i, x) # tensor([10, 0, 50, 40])
```

demo/gather.py

demo/scatter.py

Summary and Outlook

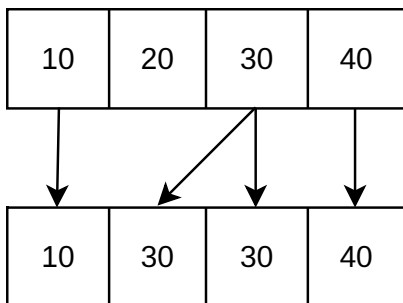
Regular data parallelism is great.

Next week: sparse and nested tensors.

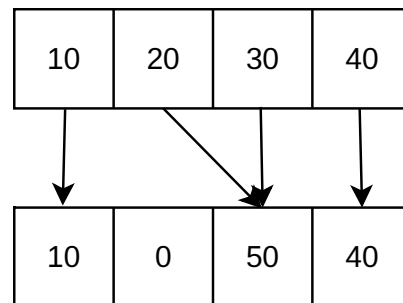
Nested Data Parallelism

Irregular data parallelism

Non-uniform access



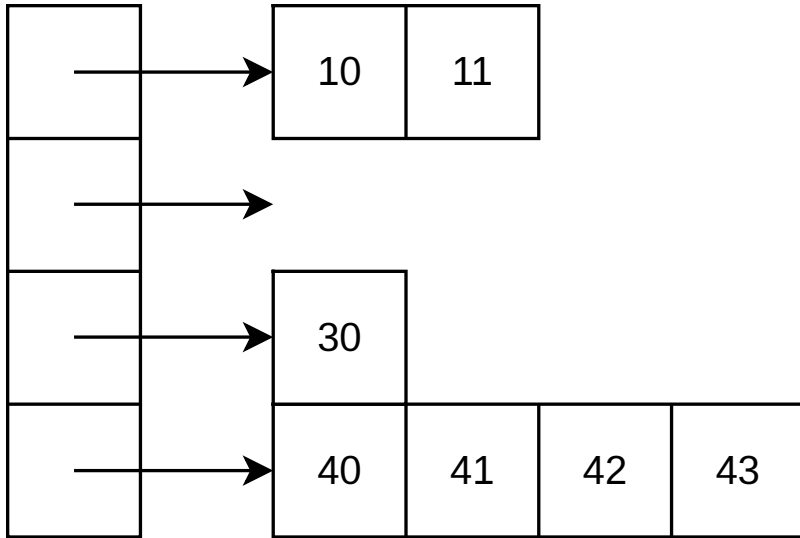
Gather



Scatter

Nested Data Parallelism

Ragged arrays

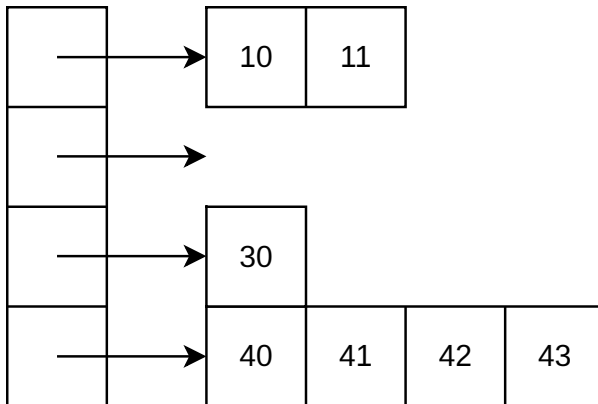


When to use it?

Whenever you have structured data with differently sized components.

Examples: sparse matrices, graph algorithms, hierarchical space divisions, document analysis, path tracing, ...

Only when saved work exceeds overhead.



10	11	0	0
0	0	0	0
30	0	0	0
40	41	42	43

Futhark

<https://futhark-lang.org/>

Futhark is a [...] statically typed, data-parallel, and purely functional array language in the ML family, and comes with a heavily optimising ahead-of-time compiler that presently generates either GPU code via CUDA and OpenCL, or multi-threaded CPU code.

```
def average (xs: []f64) = reduce (+) 0 xs / f64.i64 (length xs)
```

Designed by Troels Henriksen, Cosmin Oancea, Martin Elsmann at DIKU, from 2014

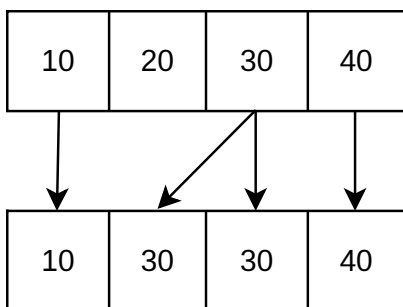
Regular Data Parallelism in Futhark

```
def scale [n] (a: [n]f64): [n]f64 =  
  map (*2) a  
  
def add [n] (a: [n]f64) (b: [n]f64): [n]f64 =  
  map2 (+) a b  
  
def factorial (n: i64): i64 =  
  reduce (*) 1 (1...n)
```

<https://futhark-lang.org/docs/prelude/>

demo/basics.fut

Gather

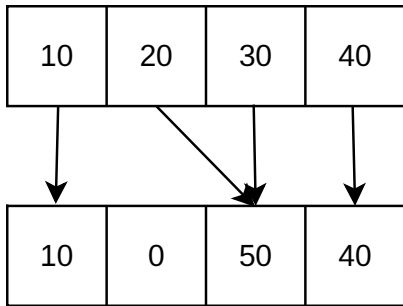


```
def gather (xs: []f64) (is: []i32) =  
  map (\i -> xs[i]) is  
  
entry main =  
  gather [10,20,30,40] [0,2,2,3]
```

demo/gather.fut

demo/gather2.fut

Scatter

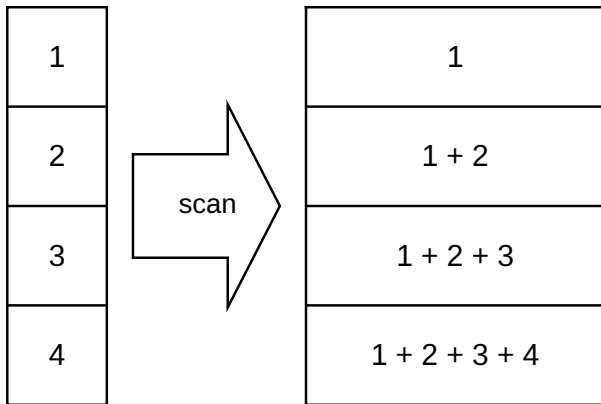


```
def scatter (is: []i64) (vs: []f64) : []f64 =  
  reduce_by_index (replicate 4 0) (+) 0 is vs  
  
entry main (is: [4]i64) (vs: [4]f64) : [4]f64 =  
  scatter is vs
```

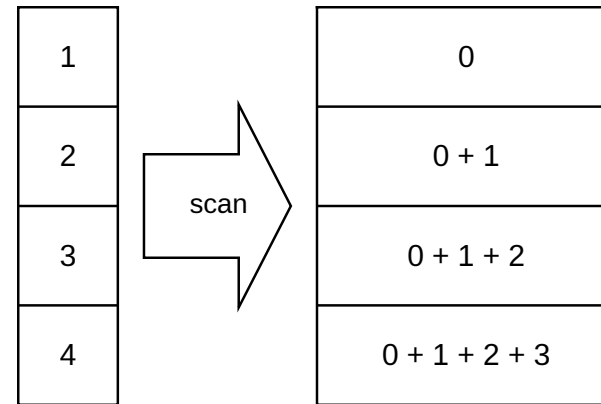
demo/scatter.fut

demo/histogram.fut

Parallel Scan



Inclusive scan



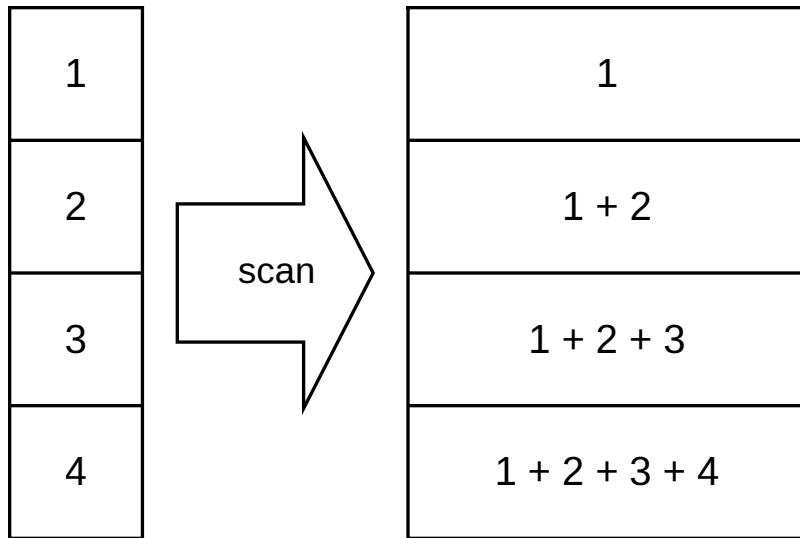
Exclusive scan

When to use it

- integral images
- parsing brackets
- sparse matrices
- radix sort
- ...

Work and Span of Parallel Scan

<http://conal.net/papers/generic-parallel-functional/>



Naive: Work $O(N^2)$, Span $O(N)$

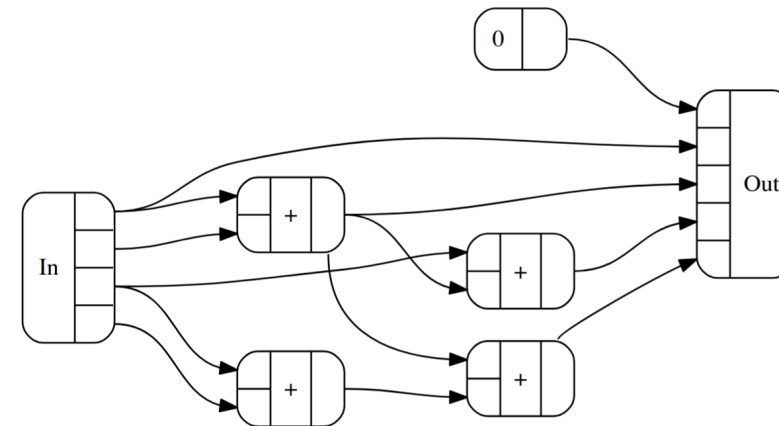


Fig. 18. *lscan* @(Bush 1) [$W=4$, $D=2$]

Conal Elliott. 2017. Generic functional parallel programming:
Scan and FFT.

Optimal: Work $O(N)$, Span $O(\log(N))$

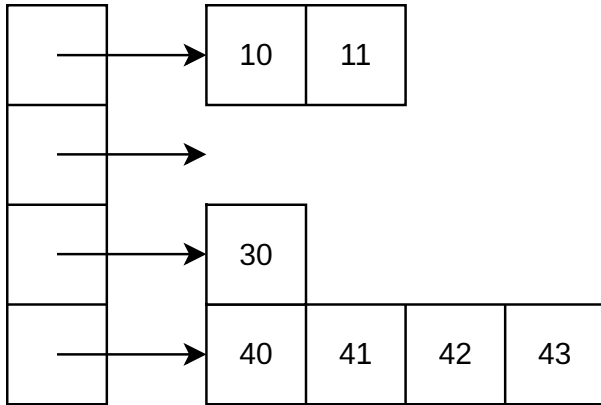
Parallel Scan in Futhark

```
def prefix_sum [n] (xs: [n]f64) : [n]f64 =  
  scan (+) 0 xs
```

demo/scan.fut

demo/counting_sort.fut

Sparse Arrays



10	11	0	0
0	0	0	0
30	0	0	0
40	41	42	43

When to use them

- Graph algorithms
- One-hot encodings
- Hessian matrices
- Linear and quadratic programming
- Sparse voxel grids
- ...

Coordinate list (COO)

10	11	0	0
0	0	0	0
30	0	0	0
40	41	42	43

Rows: [0, 0, 2, 3, 3, 3, 3]

Columns: [0, 1, 0, 0, 1, 2, 3]

Values: [10, 11, 30, 40, 41, 42, 43]

Compressed Sparse Row (CSR)

10	11	0	0
0	0	0	0
30	0	0	0
40	41	42	43

Rows: [0, 2, 2, 3, 7]

Columns: [0, 1, 0, 0, 1, 2, 3]

Values: [10, 11, 30, 40, 41, 42, 43]

ELLPACK (ELL)

10	11	0	0
0	0	0	0
30	0	0	0
40	41	42	43

Columns:

[[0, 1, -, -], [-, -, -, -], [0, -, -, -], [0, 1, 2, 3]]

Values:

[[10, 11, 0, 0], [0, 0, 0, 0], [30, 0, 0, 0], [40, 41, 42, 43]]

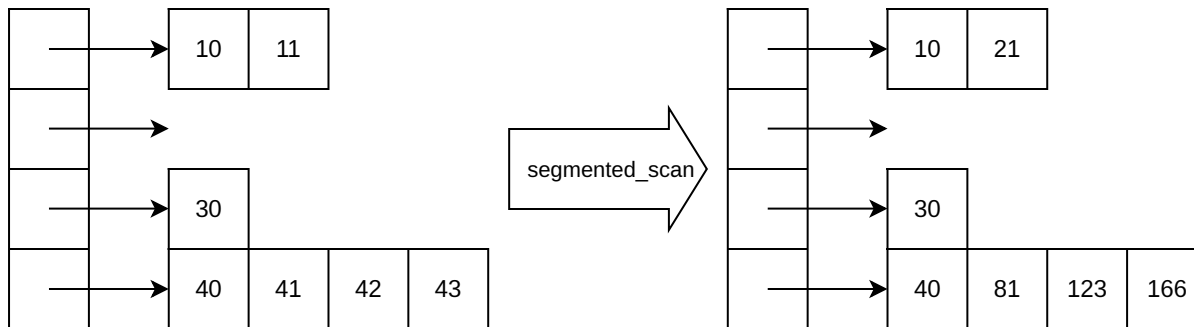
demo/coo.fut

demo/csr.fut

demo/ell.fut

Segmented Scan

Perform a scan operation on each array segment.



Example: row-wise summation.

Segmented Scan in Futhark

```
def segmented_scan_add [k] (flags: [k]bool) (values: [k]f64): [k]f64 =  
  let combine (b1, v1) (b2, v2) = (b1 || b2, if b2 then v2 else v1 + v2)  
  let pairs = scan combine (false, 0.0) (zip flags values)  
  in map (.1) pairs
```

demo/segmented.fut

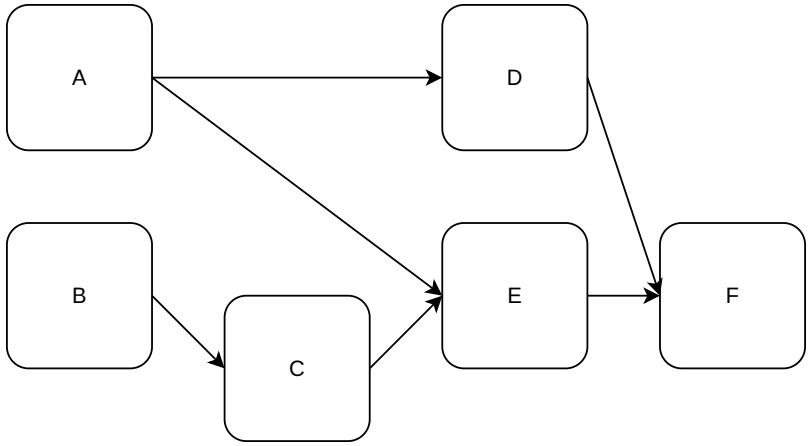
Summary and Outlook

Nested data parallelism is hard.

Next week: task parallelism.

Task Parallelism

Task Graphs



Multiple Instruction Multiple Data (MIMD) parallelism.

Execution time of tasks might be vastly different.

Graph might be static or dynamic.

When to use it?

Whenever you have heterogenous computation on heterogenous data.

Examples:

- compiler pipelines
- computer games
- unbalanced search trees
- ...

Only when regular and nested data parallelism do not apply.

Only when speedup through parallelism exceeds coordination overhead.

Java with Project Loom

<https://docs.oracle.com/en/java/javase/25/core/virtual-threads.html>

```
Thread.Builder builder = Thread.ofVirtual().name("MyThread");
Runnable task = () -> {
    System.out.println("Running thread");
};
Thread thread = builder.start(task);
System.out.println("Thread t name: " + thread.getName());
thread.join();
```

Project Loom lead by Ron Pressler, many contributors, started 2017, merged 2022.

Data Parallelism in Java

<https://docs.oracle.com/javase/tutorial/collections/streams/parallelism.html>

To create a parallel stream, invoke the operation `Collection.parallelStream`.

```
double average = roster
    .parallelStream()
    .filter(p -> p.getGender() == Person.Sex.MALE)
    .mapToInt(Person::getAge)
    .average()
    .getAsDouble();
```

Note that parallelism is not automatically faster than performing operations serially, although it can be if you have enough data and processor cores. [...], it is still your responsibility to determine if your application is suitable for parallelism.

demo/Sparse.java

Promises and Futures

The evaluator does this by creating and returning for each subexpression a *future*, which is a promise to deliver the value of that subexpression at some later time, if it has a value. Each future can evaluate its subexpression independently and concurrently with other futures because it is created with its own evaluator *process*, which is dedicated to evaluating its subexpression.

Henry C. Baker and Carl Hewitt. 1977. The incremental garbage collection of processes.

Promise: Must be written to (fulfilled, resolved, completed) exactly once.

Future: Can be read from (awaited) more than once.

In practice: a promise and a future are the same mutable reference cell.

Task Parallelism in Java

```
ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();  
  
Future<Integer> A = executor.submit(() -> 1 + 1);  
  
Future<Integer> B = executor.submit(() -> 2 + 2);  
  
Future<Integer> C = executor.submit(() -> A.get() + B.get());  
  
System.out.println(C.get());
```

<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/ExecutorService.html>

<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/Future.html>

demo/Basics.java

No Cycles when using Futures

Reading from a future suspends work until the promise is fulfilled.

Each task can only depend on futures created earlier.

```
Future<Integer> A = executor.submit(() -> B.get() + 1); // B is not in scope  
Future<Integer> B = executor.submit(() -> A.get() + 1);
```

No cycles in task graph and therefore no deadlocks are possible.

Granularity

Tasks are the unit of parallelism.

Coarse-grained tasks: low parallelism.

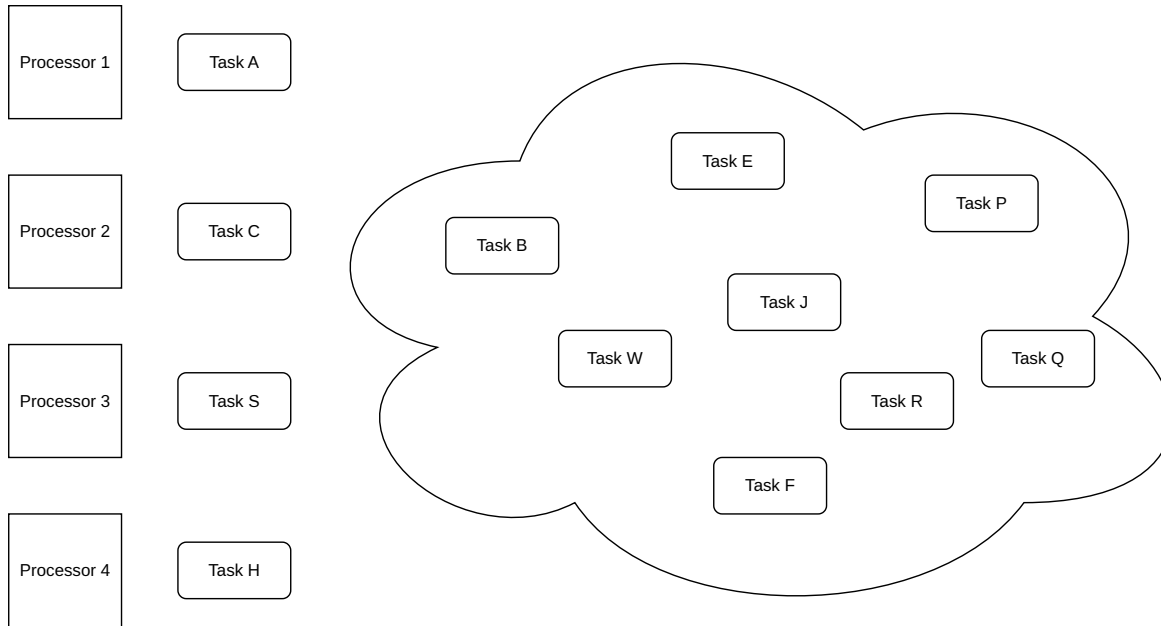
Fine-grained tasks: high overhead.

We need to balance both.

demo/Mergesort.java

Work Stealing

Work stealing attempts to make sure all processors are fully utilized.

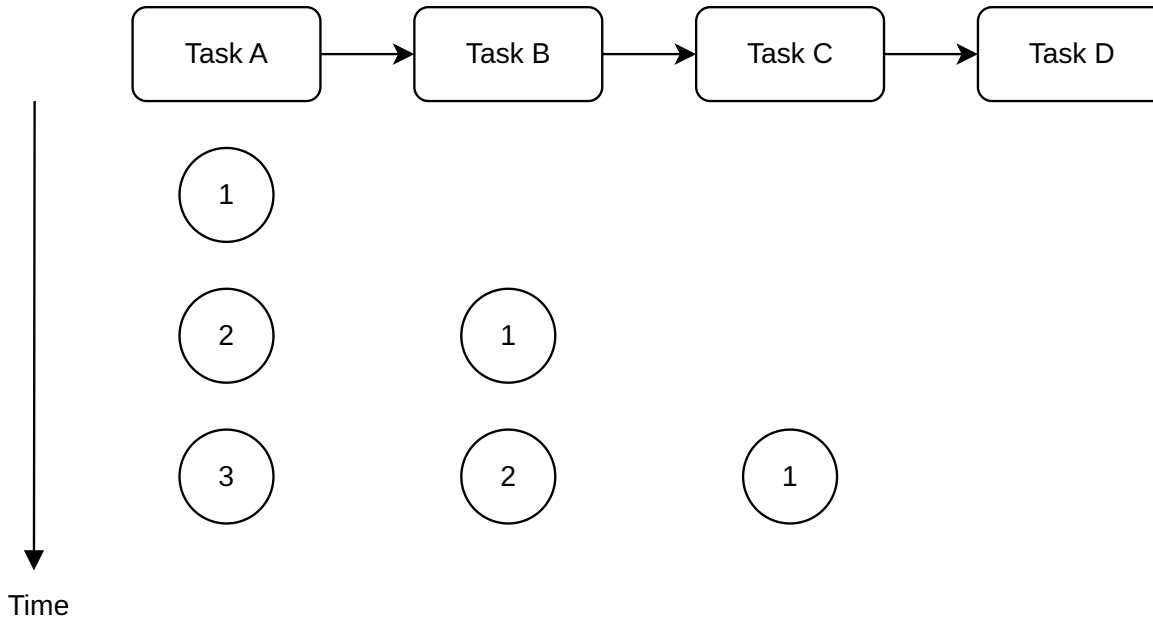


Tasks can be created dynamically and their execution time varies.

demo/SocialNetwork.java

Pipelining

When processing a stream of items, divide computation into stages.



Each stage processes items in parallel to other stages.

Blocking Queues

First-in-first-out (FIFO) data and control structure.

Used for communication and synchronization between stages.

Can in theory be unbounded, should in practice be bounded.

Consumer blocks when empty, producer blocks when full.

Blocking Queues in Java

```
BlockingQueue<String> queue = new ArrayBlockingQueue<>(10);  
  
queue.put("Hello");  
queue.put("World");  
  
String hello = queue.take();  
String world = queue.take();
```

<https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/BlockingQueue.html>

<https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/ArrayBlockingQueue.html>

<https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/LinkedBlockingQueue.html>

Pipelining in Java

```
BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(1);  
executor.execute(() -> producer(queue));  
executor.execute(() -> consumer(queue));
```

```
static void producer(BlockingQueue<Integer> queue) {  
    try {  
        for (int i = 0; i < 5; i++) queue.put(i);  
    } catch (InterruptedException e) {}  
}  
  
static void consumer(BlockingQueue<Integer> queue) {  
    try {  
        for (int i = 0; i < 5; i++) System.out.println(queue.take() * 2);  
    } catch (InterruptedException e) {}  
}
```

"Execution in the Kingdom of Nouns"

demo/Pipeline.java

Cycles when using Queues

```
BlockingQueue<Integer> queue1 = new ArrayBlockingQueue<>(1);  
BlockingQueue<Integer> queue2 = new ArrayBlockingQueue<>(1);  
executor.execute(() -> { queue2.put(queue1.take() * 2); });  
executor.execute(() -> { queue1.put(queue2.take() + 1); });
```

Cyclic dependencies cause deadlocks.

demo/Deadlock.java

Poison Pill Pattern

Mark the end of the stream with a sentinel value.

Signal from producer to consumer that it should stop.

One producer, multiple consumers: send one poison pill for each consumer.

Multiple producers, one consumer: count number of poison pills.

How can the consumer signal to the producer to stop?

Poison Pill in Java

```
// producer
for (int i = 0; i < 5; i++) {
    queue.put(Optional.of(i));
}
queue.put(Optional.empty());
```

```
// consumer
while (true) {
    Optional<Integer> value = queue.take();
    if (value.isEmpty()) break;
    System.out.println(value.get());
}
```

demo/PoisonPill.java

Batching

Accumulate items, process entire batches.

Processing a batch might be more efficient: work sharing, data parallelism, ...

Pay in latency, gain in throughput.

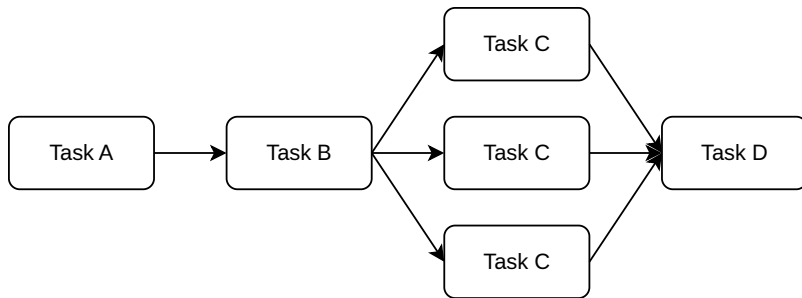
demo/Batching.java

Balancing

In a pipeline, each stage should take roughly the same amount of time.

Blocking queues naturally apply backpressure when consumer cannot keep up.

We can also assign multiple tasks to the same stage for balancing.



The order of items is not preserved.

demo/Balancing.java

Input and Output

Computation on other devices and machines can happen in parallel.

Example: fetch data from a database.

```
Future<Integer> A = executor.submit(() -> fetch(8001));  
Future<Integer> B = executor.submit(() -> fetch(8002));  
System.out.println(A.get() + B.get());
```


demo/Client.java

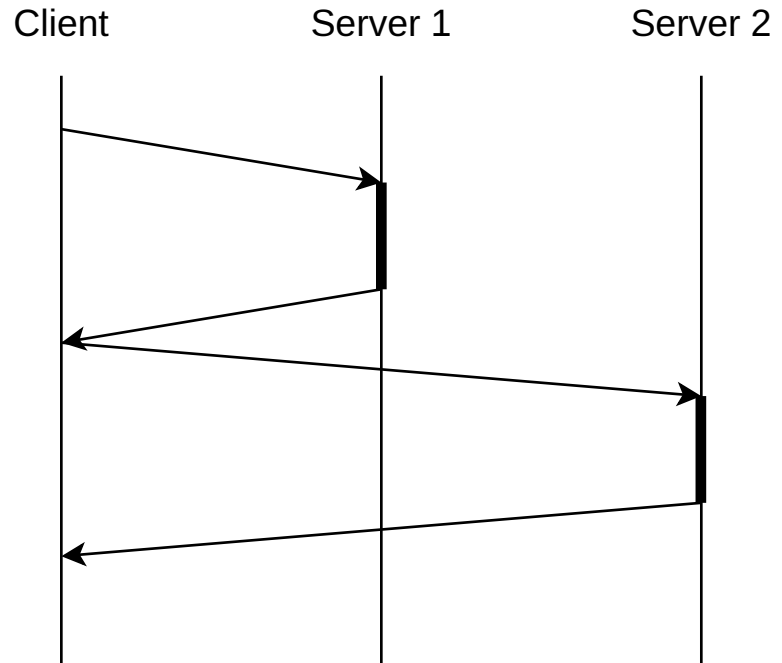
Summary and Outlook

Task parallelism is flexible.

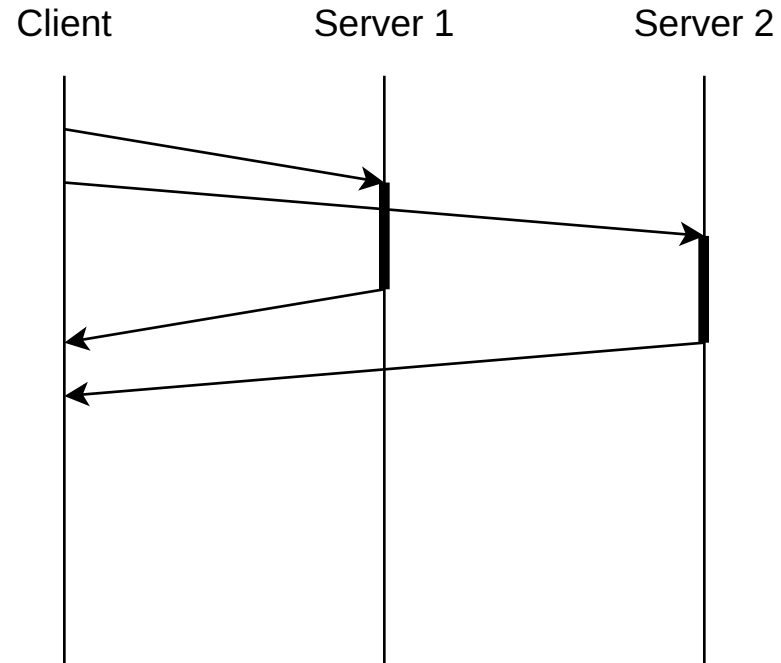
Next week: Asynchronous communication.

Asynchronous Communication

Avoid Blocking



Synchronous



Asynchronous

When to use it

Parallelism across different machines (drives, servers, ...)

Input and output (files, network, ...)

Only when there is more than one device!

JavaScript with Node.js

<https://developer.mozilla.org/en-US/docs/Web/JavaScript>

```
import { createServer } from 'node:http';

let server = createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello World!\n');
});

server.listen(3000, '127.0.0.1', () => {
  console.log('Listening on 127.0.0.1:3000');
});
```

JavaScript originally designed by Brendan Eich, around 1995

Node.js originally created by Ryan Dahl, 2009

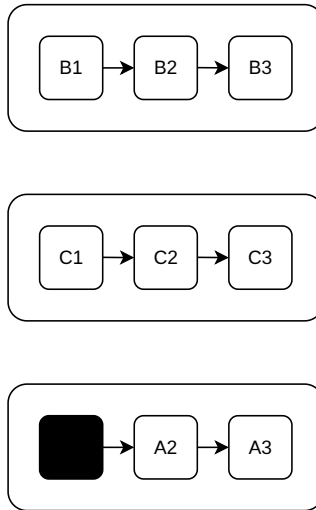
demo/server1.js

Eventloop

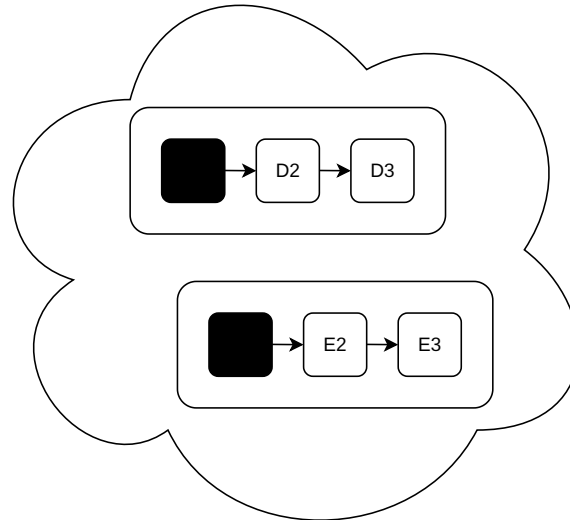
Program



Task Queue



Operating System

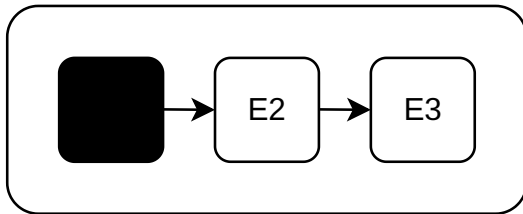
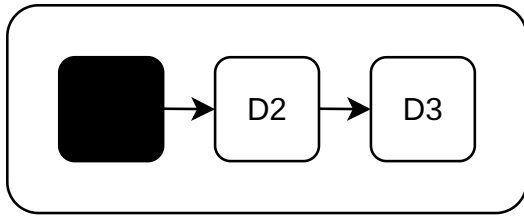


Runs on a single processor.

Push-based, not pull-based.

Continuations (Callbacks)

Instructions on how to continue.



Inversion of control. "Don't call us, we call you."

Continuation-Passing Style (CPS)

```
function add(a, b, k) {  
  k(a + b);  
}  
  
function multiply(a, b, k) {  
  k(a * b);  
}  
  
function multiply_add(x, k) {  
  // 3 * x + 5  
  multiply(3, x, y => add(y, 5, k));  
}  
  
multiply_add(4, r => console.log(r));
```

demo/continuations.js

CPS for Asynchronous Communication

```
setTimeout(() => console.log('A done'), 1000);  
setTimeout(() => console.log('B done'), 2000);
```

```
setTimeout(() => {  
  console.log('A done');  
  setTimeout(() => {  
    console.log('B done');  
  }, 2000);  
, 1000);
```

demo/timing.js

demo/callback_client.js

Error Handling with Continuations

Exceptions in callbacks cannot be caught.

A common pattern is to pass an error handling callback.

demo/filesystem.js

Promises in Node.js

`Promise.resolve` enqueues a new task, `.then` defines task steps.

```
console.log('START');

Promise.resolve()
  .then(() => console.log('A1'))
  .then(() => console.log('A2'));

Promise.resolve()
  .then(() => console.log('B1'))
  .then(() => console.log('B2'));

console.log('END');
```

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

demo/eventloop.js

demo/promise_client.js

Fulfilling Promises

A Promise represents a future value: it may be pending, fulfilled, or rejected.

A new promise starts as pending.

Fulfilling a promise means giving it a value.

Rejecting a promise means giving it an error value.

Fulfilling Promises in JavaScript

```
let A = new Promise((resolve) => {
  fs.readFile('input1.txt', 'utf8', (err, data) => {
    resolve(data);
  });
});

let B = new Promise((resolve) => {
  fs.readFile('input2.txt', 'utf8', (err, data) => {
    resolve(data);
  });
});

A.then((a) => {
  B.then((b) => {
    console.log(a, b);
  });
});
```

demo/fulfill.js

Error Handling with Promises

Promises propagate the error to the conceptual caller.

```
let A = new Promise((resolve, reject) => {
  fs.readFile('input1.txt', 'utf8', (err, data) => {
    if (err) reject(err);
    else resolve(data);
  });
});

A.then((a) => {
  B.then((b) => {
    console.log(a, b);
  });
}).catch((err) => {
  console.log('Error:', err);
});
```

demo/errors.js

Async Await Syntax

Transform this:

```
async function f() {  
  let a = fetch("a.net");  
  let b = fetch("b.net");  
  return (await a) + (await b);  
}
```

Into:

```
function f() {  
  let a = fetch("a.net");  
  let b = fetch("b.net");  
  return a.then( (x) =>  
    b.then( (y) =>  
      Promise.resolve(x + y)  
    )  
  );  
}
```

Async Syntax in JavaScript

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function

The async function declaration creates a binding of a new async function to a given name. The `await` keyword is permitted within the function body, enabling asynchronous, promise-based behavior to be written in a cleaner style and avoiding the need to explicitly configure promise chains.

```
import * as fs from 'node:fs/promises';

let a = fs.readFile('input1.txt', 'utf8');
let b = fs.readFile('input2.txt', 'utf8');

console.log(await a, await b);
```

demo/await.js

Error Handling with Async Syntax

Error is rethrown at await point.

```
import * as fs from 'node:fs/promises';

try {
  let a = fs.readFile('input1.txt', 'utf8');
  let b = fs.readFile('input2.txt', 'utf8');
  console.log(await a, await b);
} catch (err) {
  console.log('Error:', err);
}
```

demo/await_errors.js

demo/await_client.js

Async Generators in JavaScript

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function*

```
async function* asyncCounter() {  
  for (let i = 0; i < 4; i++) {  
    await sleep(1000);  
    yield i;  
  }  
}
```

Asynchronous User Interfaces in JavaScript

Waiting for communication should not freeze the user interface.

```
for (let i = 0; i < 1e8; i++) {  
  counter.textContent = i;  
}
```

After updating the document we should yield control to the browser.

```
let i = 0;  
function step() {  
  counter.textContent = i;  
  i++;  
  if (i < 1e8) setTimeout(step, 0);  
}
```


demo/freeze.html

Races

Racing two promises for completion gives a non-deterministic result.

```
let timeout = new Promise((_, reject) =>
  setTimeout(() => reject(new Error('timeout')), 1000)
);

let request = fetch('http://localhost:8001')
  .then(response => response.text());

Promise.race([request, timeout]).then(result => {
  console.log(result);
}).catch(err => {
  console.log(err.message);
});
```

Now we are starting to talk about concurrency.

Summary and Outlook

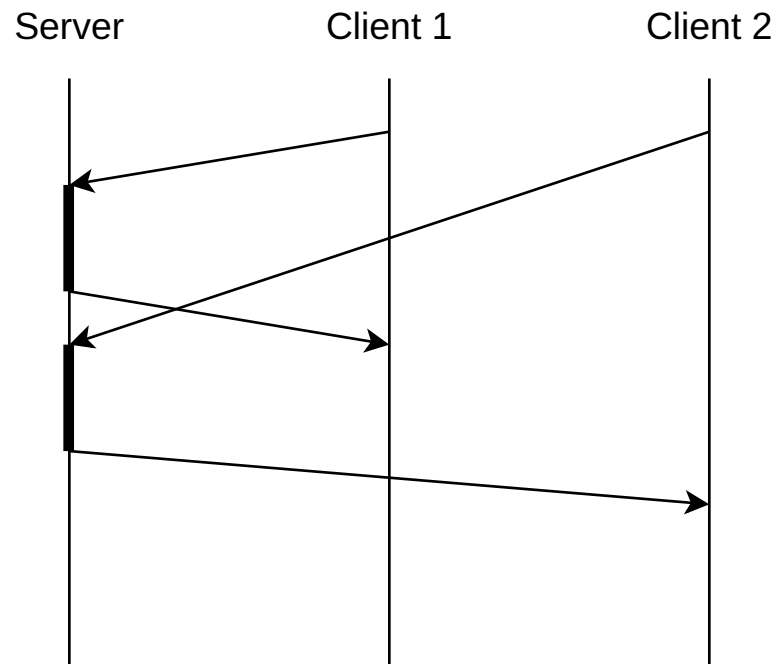
Asynchronous communication enables parallelism across devices.

Next week: Channels.

Communicating Processes

Concurrency

Events happen in unpredictable order.

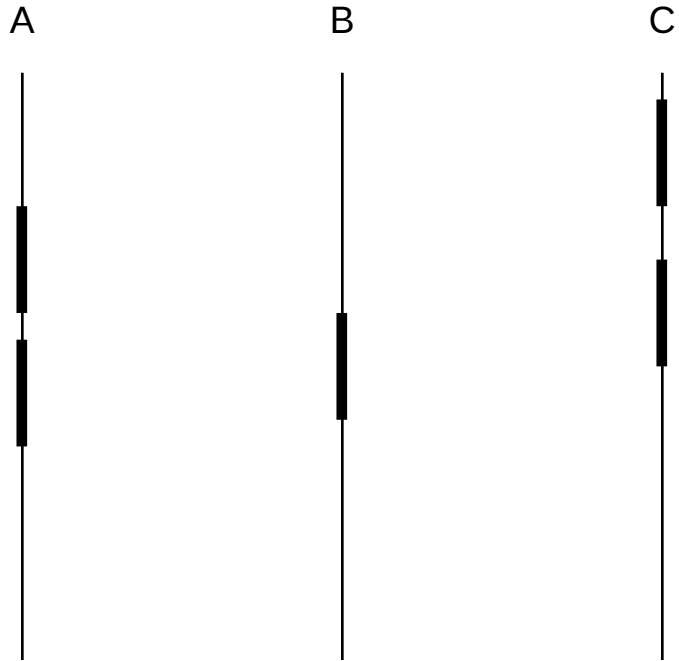


How do we model this fact in programs?

Examples:

- Http Servers
- File Servers
- Chat Serves
- Stream Processors
- Event Queues
- Monitors
- Dashboards
- "Internet of Things"

Processes



Furthermore, we have stipulated that the processes should be connected loosely; by this we mean that apart from the (rare) moments of explicit intercommunication, the individual processes themselves are to be regarded as completely independent of each other. In particular we disallow any assumption about the relative speeds of the different processes.

Dijkstra, E. W. 1965. Cooperating Sequential Processes.

Go

<https://go.dev/>

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, 世界")
}
```

Designed by Robert Griesemer, Rob Pike, Ken Thompson. First appeared 2009.

Processes in Go

```
go func() { fmt.Println("A1"); fmt.Println("A2") }()  
go func() { fmt.Println("B1"); fmt.Println("B2") }()  
go func() { fmt.Println("C1"); fmt.Println("C2") }()
```

demo/processes/main.go

Communication is synchronization

Happens Before:

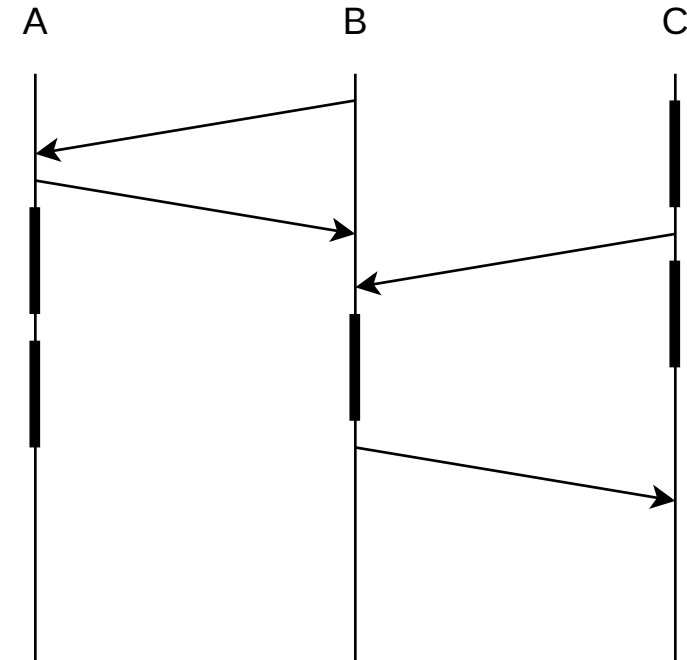
(1) If a and b are events in the same process, and a comes before b , then $a \rightarrow b$.

(2) If a is the sending of a message by one process and b is the receipt of the same message by another process, then $a \rightarrow b$.

(3) If $a \rightarrow b$ and $b \rightarrow c$ then $a \rightarrow c$.

Two distinct events a and b are said to be concurrent if $a \nrightarrow b$ and $b \nrightarrow a$.

Leslie Lamport. 1978. Time, clocks, and the ordering of events in a distributed system.



Rendezvous Channels

Such communication occurs when one process names another as destination for output *and* the second process names the first as source for input. In this case, the value to be output is copied from the first process to the second. There is *no* automatic buffering: In general, an input or output command is delayed until the other process is ready with the corresponding output or input.

C. A. R. Hoare. 1978. Communicating sequential processes.

Rendezvous Channels in Go

https://go.dev/ref/spec#Channel_types

https://go.dev/ref/spec#Send_statements

https://go.dev/ref/spec#Receive_operator

```
c := make(chan int)
c <- 1
x := <-c
```

```
c := make(chan int)
go func() { c <- 1 }()
x := <-c
```

demo/channels/main.go

demo/client/main.go

Closing Channels in Go

<https://pkg.go.dev/builtin#close>

```
func close(c chan<- Type)
```

The close built-in function closes a channel, which must be either bidirectional or send-only. It should be executed only by the sender, never the receiver, and has the effect of shutting down the channel after the last sent value is received. After the last value has been received from a closed channel `c`, any receive from `c` will succeed without blocking, returning the zero value for the channel element. The form

```
x, ok := <-c
```

will also set `ok` to false for a closed and empty channel.

demo/pipeline_consumers/main.go

WaitGroups in Go

<https://pkg.go.dev/sync#WaitGroup>

A WaitGroup is a counting semaphore typically used to wait for a group of goroutines or tasks to finish.

For example:

```
var wg sync.WaitGroup
wg.Go(task1)
wg.Go(task2)
wg.Wait()
```

A WaitGroup may also be used for tracking tasks without using Go to start new goroutines by using WaitGroup.Add and WaitGroup.Done.

demo/pipeline_producers/main.go

Guarded Commands

A guarded command with an input guard is selected for execution only if and when the source named in the input command is ready to execute the corresponding output command. If several input guards of a set of alternatives have ready destinations, only one is selected and the others have no effect; but the choice between them is arbitrary.

C. A. R. Hoare. 1978. Communicating sequential processes.

Guarded Commands in Go

https://go.dev/ref/spec#Select_statements

A "select" statement chooses which of a set of possible send or receive operations will proceed. It looks similar to a "switch" statement but with the cases all referring to communication operations.

```
select {  
case text := <-input:  
    fmt.Println("You entered:", text)  
case <-timeout:  
    fmt.Println("Timeout!")  
}
```

demo/select/main.go

demo/chat_client/main.go

demo/chat_server1/main.go

demo/chat_server2/main.go

Race Condition

Originally, a term coming from hardware.

In Software: when the outcome of a program depends on the timing of processes.

Can be intentional or unintentional.

Examples seen so far: channels and select.

Data Race

What happens when two processes modify a shared mutable reference?

```
x1 = r.read  
y1 = x1 + 1  
r.write(y1)
```

```
x2 = r.read  
y2 = x2 + 1  
r.write(y2)
```

Some programming languages provide no guarantee at all about such programs.

<https://go.dev/ref/mem>

demo/races/main.go

```
go run -race
```

Mutexes

To begin, consider N computers, each engaged in a process which, for our aims, can be regarded as cyclic. In each of the cycles a so-called "critical section" occurs and the computers have to be programmed in such a way that at any moment only one of these N cyclic processes is in its critical section. In order to effectuate this mutual exclusion of critical-section execution the computers can communicate with each other via a common store.

E. W. Dijkstra. 1965. Solution of a problem in concurrent programming control.

Mutexes in Go

<https://pkg.go.dev/sync#Mutex>

```
func (m *Mutex) Lock()
```

Lock locks m. If the lock is already in use, the calling goroutine blocks until the mutex is available.

```
func (m *Mutex) Unlock()
```

Unlock unlocks m. It is a run-time error if m is not locked on entry to Unlock.

demo/races/main.go

demo/chat_sever3/main.go

Deadlocks

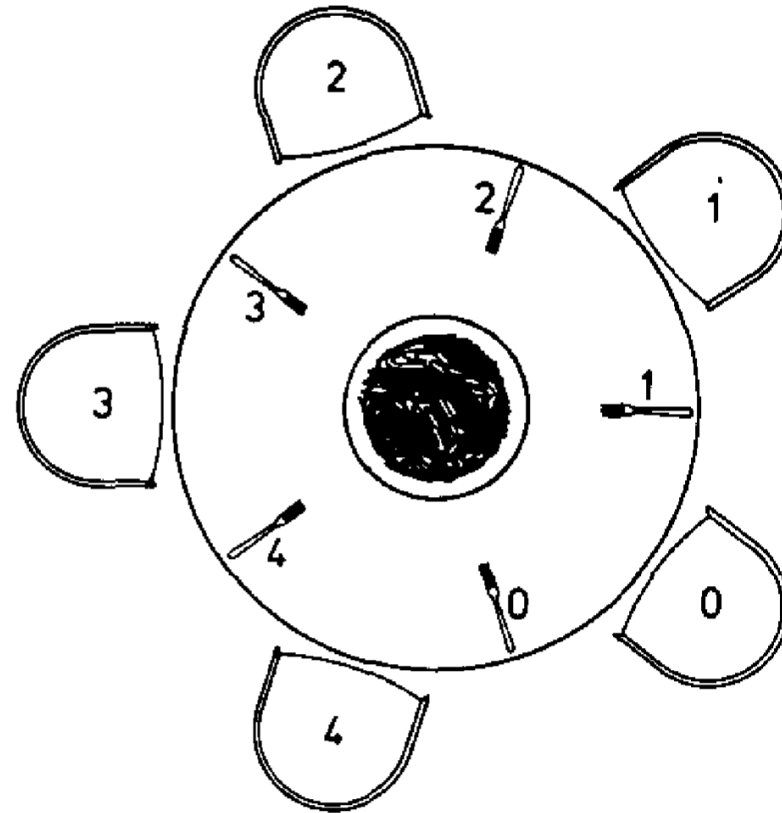
One of the objectives of recent developments in operating systems – incorporating multi-programming, multiprocessing, etc. – has been to improve the utilization of system resources (and hence reduce the cost to users) by distributing them among many concurrently executing tasks. In any operating system of this type the problem of deadlock must be considered. Requests by separate tasks for resources may possibly be granted in such a sequence that a group of two or more tasks is unable to proceed - each task monopolizing resources and waiting for the release of resources currently held by others in that group.

E. G. Coffman, M. Elphick, and A. Shoshani. 1971. System Deadlocks.

Dining Philosophers

Five philosophers spend their lives thinking and eating. The philosophers share a common dining room where there is a circular table surrounded by five chairs, each belonging to one philosopher. In the center of the table there is a large bowl of spaghetti, and the table is laid with five forks (see Figure 1). On feeling hungry, a philosopher enters the dining room, sits in his own chair, and picks up the fork on the left of his place. Unfortunately, the spaghetti is so tangled that he needs to pick up and use the fork on his right as well. When he has finished, he puts down both forks, and leaves the room.

Fig. 1.



Dining Philosophers in Go

Philosophers are goroutines.

```
go func() {  
    fork0.Lock();  
    fork1.Lock();  
    fmt.Println("eating");  
    fork0.Unlock();  
    fork1.Unlock();  
    wg.Done();  
}()
```

Forks are mutexes: picking them up is locking, putting them down is unlocking.

demo/dining/main.go

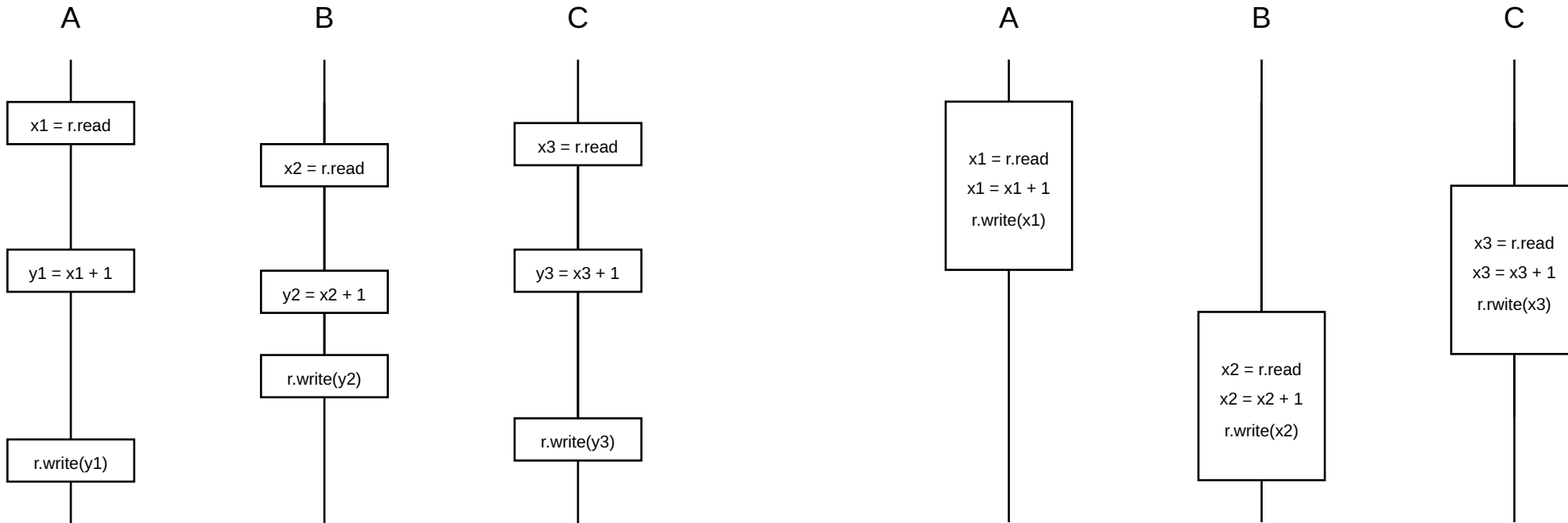
Summary and Outlook

Communicating processes model concurrency.

Next week: transactional memory.

Transactional Memory

Transactions



Alternative to manual locking.

Still non-deterministic, but interleaves only between transactions.

Serializability

We can broaden the class of allowable executions to include executions that have the same effect as serial ones. Such executions are called serializable. More precisely, an execution is serializable if it produces the same output and has the same effect on the database as some serial execution of the same transactions.

Philip A. Bernstein, Vassco Hadzilacos, and Nathan Goodman. 1987. Concurrency control and recovery in database systems.

Opacity

Opacity is a safety property that captures the intuitive requirements that

- (1) all operations performed by every committed transaction appear as if they happened at some single, indivisible point during the transaction lifetime,
- (2) no operation performed by any aborted transaction is ever visible to other transactions (including live ones), and
- (3) every transaction always observes a consistent state of the system.

Rachid Guerraoui and Michal Kapalka. 2008. On the correctness of transactional memory.

Haskell with the Glasgow Haskell Compiler

<https://www.haskell.org/>

<https://www.haskell.org/ghc/>

```
primes = filterPrime [2..] where
  filterPrime (p:xs) =
    p : filterPrime [x | x <- xs, x `mod` p /= 0]
```

Glasgow Haskell Compiler by Simon Peyton-Jones, Simon Marlow and others. First appeared 1992.

demo/Basics.hs

Channels in Haskell

<https://hackage.haskell.org/package/base/docs/Control-Concurrent-Chan.html>

```
newChan :: IO (Chan a)
writeChan :: Chan a -> a -> IO ()
readChan :: Chan a -> IO a
```

demo/Pipeline.hs

MVars in Haskell

<https://hackage.haskell.org/package/base/docs/Control-Concurrent-MVar.html>

```
newEmptyMVar :: IO (MVar a)
newMVar     :: a -> IO (MVar a)
takeMVar    :: MVar a -> IO a
putMVar     :: MVar a -> a -> IO ()
```

demo/Dining.hs

Select in Haskell

```
race :: MVar a -> MVar b -> IO (Either a b)
race mvar1 mvar2 = do
    result <- newEmptyMVar
    forkIO do
        value1 <- takeMVar mvar1
        tryPutMVar result (Left value1)
        return ()
    forkIO do
        value2 <- takeMVar mvar2
        tryPutMVar result (Right value2)
        return ()
    takeMVar result
```


demo/Timeout.hs

Software Transactional Memory (STM)

A transaction is a finite sequence of local and shared memory machine instructions:

Read_transactional - reads the value of a shared location into a local register.

Write_transactional – stores the contents of a local register into a shared location.

[...]

Any transaction may either fail, or complete successfully, in which case its changes are visible atomically to other processes.

Nir Shavit and Dan Touitou. 1995. Software transactional memory.

Software Transactional Memory in Haskell

Concurrent programming is notoriously tricky. Current lock-based abstractions are difficult to use and make it hard to design computer systems that are reliable and scalable. Furthermore, systems built using locks are difficult to compose without knowing about their internals.

Tim Harris, Simon Marlow, Simon Peyton-Jones, and Maurice Herlihy. 2005. Composable memory transactions.

```
-- Transactional variables
data TVar a
newTVar :: a -> STM (TVar a)
readTVar :: TVar a -> STM a
writeTVar :: TVar a -> a -> STM ()

-- Running STM computations
atomically :: STM a -> IO a
retry :: STM a
orElse :: STM a -> STM a -> STM a

-- The STM monad itself
data STM a
instance Monad STM
-- Monads support "do" notation and sequencing
```

demo/ReadWrite.hs

demo/Swap.hs

Transactional Data Structures

There are two valid approaches:

1. Immutable data structure in a transactional variable.
2. Mutable data structure using transactional variables.

There is one invalid approach:

3. Mutable data structure behind a transactional variable.

The API's caller remains responsible for ensuring scalability by making it unlikely that concurrent operations will need to modify overlapping sets of locations. However, this is a performance problem rather than a correctness or liveness one, and in our experience, even straightforward data structures, developed directly from sequential code, offer performance which competes with and often surpasses state-of-the-art lock-based designs.

Transactional Data Structures in Haskell

1. Immutable data structure in a transactional variable.

```
type Queue a = ([a], [a])  
type TQueue a = TVar (Queue a)
```

2. Mutable data structure using transactional variables.

```
data Node a = Nil | Node a (TVar (Node a))  
type TQueue a = (TVar (TVar (Node a)), TVar (TVar (Node a)))
```

3. Mutable data structure behind a transactional variable.

Highly discouraged and unlikely in Haskell.

demo/BankersQueue.hs

demo/MutableQueue.hs

Aborting Transactions

Sometimes, we want to abort a transaction based on a user-defined condition.

```
retry :: STM a
```

Conceptually, `retry` aborts the transaction with no effect, and restarts it at the beginning. However, there is no point in actually re-executing the transaction until at least one of the TVars read during the attempted transaction is written by another thread.

Tim Harris, Simon Marlow, Simon Peyton-Jones, and Maurice Herlihy. 2005. Composable memory transactions.

demo/DiningTransaction.hs

Selecting Transactions

Sometimes, we want to select the first transaction that succeeds.

```
orElse :: STM a -> STM a -> STM a
```

The transaction `s1 'orElse' s2` first runs `s1`; if it retries, then `s1` is abandoned with no effect, and `s2` is run. If `s2` retries as well, the entire call retries — but it waits on the variables read by either of the two nested transactions.

Tim Harris, Simon Marlow, Simon Peyton-Jones, and Maurice Herlihy. 2005. Composable memory transactions.

demo/TimeoutTransaction.hs

Progress Conditions

A method is *wait-free* if it guarantees that every call finishes its execution in a finite number of steps.

A method is *lock-free* if it guarantees that infinitely often some method call finishes in a finite number of steps.

A method is *obstruction-free* if, from any point after which it executes in isolation, it finishes in a finite number of steps

Liveness Properties of Haskell STM

The STM implementation guarantees that one transaction can force another to abort only when the first one commits. As a result, the STM implementation is lock-free in the sense that it guarantees at any time that some running transaction can successfully commit. [...] Starvation is possible.

Tim Harris, Simon Marlow, Simon Peyton-Jones, and Maurice Herlihy. 2005. Composable memory transactions.

demo/Livelock.hs

Data Versioning

In implementations, there are two main approaches to data versioning:

1. Direct Update (eager): keep an undo log.
Upon commit do nothing, upon abort undo writes.
2. Deferred Update (lazy): keep a write buffer.
Upon commit perform writes, upon abort do nothing.

In both we keep some auxiliary data structure for each transaction and execute extra code for operations on variables.

Tradeoff: properties, performance, difficulty.

Conflict Detection

In implementations, there are two main approaches to conflict detection:

1. Pessimistic: detect conflicts early, when writing to a variable.
2. Optimistic: detect conflicts late, when attempting to commit.

In both we keep set of variables read and a set of variables written for each transaction.

Tradeoff: properties, performance, difficulty.

Atomic Operations

Processors offer a variety of atomic read-write-modify operations:

Test-And-Set (TAS)

Fetch-And-Add (FAA)

Compare-And-Swap (CAS)

Load-Linked/Store-Conditional (LL/SC)

These are required for the implementation of locks and transactions.

Beware of the ABA problem with Compare-And-Swap.

Beware of livelocks with Load-Linked/Store-Conditional.

demo/Atomsics.hs

Summary and Outlook

Transactional memory data structures and algorithms are composable.

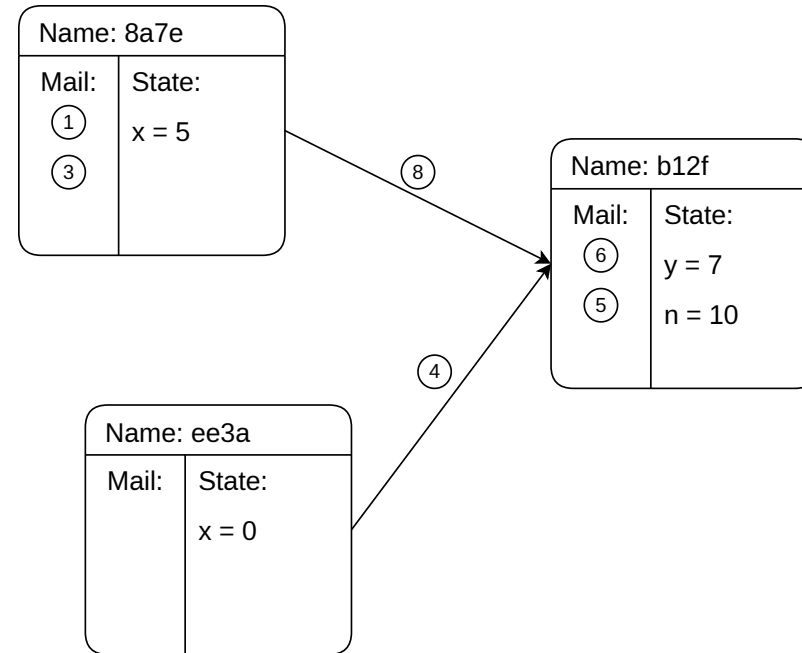
Next week: Actors

Actors

Actors Today

An actor has:

- a unique name
- a local state
- an unbounded mailbox
- a description of its behavior



The ACTOR Formalism

This paper proposes a modular ACTOR architecture and definitional method for artificial intelligence that is conceptually based on a single kind of object: actors [or, if you will, virtual processors, activation frames, or streams].

[...]

The architecture will efficiently run the coming generation of PLANNER-like artificial intelligence languages including those requiring a high degree of parallelism.

[...]

The architecture is general with respect to control structure and does not have or need goto, interrupt, or semaphore primitives.

Intuitively, an ACTOR is an active agent which plays a role on cue according to a script.

[...]

Data structures, functions, semaphores, monitors, ports, descriptions, Quillian nets, logical formulae, numbers, identifiers, demons, processes, contexts, and data bases can all be shown to be special cases of actors.

Patterns of Passing Messages

In this paper we present an approach to modelling intelligence in terms of a society of communicating knowledge-based problem-solving experts.

At a more superficial and imprecise level, each actor may be thought of as having two aspects which together realize the behavior which it manifests:

the ACTION it should take when it is sent a message;

its ACQUAINTANCES which is the finite collection of actors that it directly KNOWS ABOUT.

Carl Hewitt. 1977. Viewing control structures as patterns of passing messages.

Actors: a model of concurrent computation

An actor may be described by specifying:

- its mail address, to which there corresponds a sufficiently large mail queue; and,
- its behavior, which is a function of the communication accepted.

Actors are computational agents which map each incoming communication to a 3-tuple consisting of:

1. a finite set of communications sent to other actors;
2. a new behavior (which will govern the response to the next communication processed); and,
3. a finite set of new actors created

Gul Agha. 1986. Actors: a model of concurrent computation in distributed systems.

When to use them

- Distributed systems
- Servers and services
- Telecom and signalling systems
- Chat and messaging systems
- Event-driven microservices
- Online gaming backends

Elixir

<https://elixir-lang.org>

```
"Elixir" |> String.graphemes() |> Enum.frequencies()
```

Designed by José Valim, first appeared 2012

<https://www.erlang.org>

```
fact(0) -> 1;  
fact(N) -> N * fact(N-1).
```

```
example:fact(10).
```

Based on Erlang, designed by Joe Armstrong, started 1986, open-sourced 1998

demo/basics.exs

demo/filesystem.exs

Actors in Erlang/Elixir

Two processes operating on the same machine must be as independent as if they ran on physically separated machines.

[...]

1. Processes have "share nothing" semantics. This is obvious since they are imagined to run on physically separated machines.
2. Message passing is the only way to pass data between processes. [...]
3. Isolation implies that message passing is asynchronous. [...]
4. Since nothing is shared, everything necessary to perform a distributed computation must be copied. [...]

Joe Armstrong. 2003. Making reliable distributed systems in the presence of software errors.

demo/processes.exs

Names of Actors

We require that the names of processes are unforgeable. This means that it should be impossible to guess the name of a process, and thereby interact with that process. We will assume that processes know their own names, and that processes which create other processes know the names of the processes which they have created.

[...]

In many primitive religions it was believed that humans had powers over spirits if they could command them by their real names. Knowing the real name of a spirit gave you power over the spirit, and using this name you could command the spirit to do various things for you

Joe Armstrong. 2003. Making reliable distributed systems in the presence of software errors.

PIDs in Elixir

<https://hexdocs.pm/elixir/Kernel.html#self/0>

```
self()
```

Returns the PID (process identifier) of the calling process.

<https://hexdocs.pm/elixir/Kernel.html#spawn/1>

```
spawn(fun)
```

Spawns the given function and returns its PID.

demo/names.exs

Message Passing

Message passing obeys the following rules:

1. Message passing is assumed to be atomic which means that a message is either delivered in its entirety or not at all.
2. Message passing between a pair of processes is assumed to be ordered meaning that if a sequence of messages is sent and received between any pair of processes then the messages will be received in the same order they were sent.
3. Messages should not contain pointers to data structures contained within processes — they should only contain constants and/or Pids.

We say that such message passing has *send and pray semantics*. We *send* the message and *pray* that it arrives.

Message Passing in Elixir

<https://hexdocs.pm/elixir/Kernel.html#send/2>

```
send(dest, message)
```

Sends a message to the given dest and returns the message.

<https://hexdocs.pm/elixir/Kernel.SpecialForms.html#receive/1>

```
receive(args)
```

Checks if there is a message matching any of the given clauses in the current process mailbox.

demo/messages.exs

demo/order.exs

Message Buffers are Unbounded

In a purely physical context, the finiteness of the universe suggests that a communication sent ought to be delivered.

[...]

There are, realistically, no unbounded buffers in the physically realizable universe. This is similar to the fact that there are no unbounded stacks in the universe, and certainly not in our processors, and yet we parse recursive control structures in algolic languages as though there were an infinite stack.

The alternate to assuming unbounded space is that we have to assume some specific finite limit; but each finite limit leads to a different behavior. There is, however, no general limit on buffers: the size of any real buffer will be specific to any particular implementation and its limitations.

Gul Agha. 1986. Actors: a model of concurrent computation in distributed systems.

demo/overflow.exs

Changing State

We tail-call the actor with another argument.

```
def loop(count) do
  receive do
    :increment ->
      loop(count + 1)
  end
end
```

demo/increment.exs

demo/state.exs

Reply-to Addresses

Often the envelope of a messenger is a REQUEST which In addition to a request message contains an actor *c* to which a reply to the request should be sent. Such an envelope is packaged as follows:

(request: the-message (reply-to: *c*))

The ACTOR *c* Is closely related to the continuation FUNCTIONS used by Morris, Wadsworth, Reynolds, and Strachey.

An ordinary functional call to a function *f* with arguments *arg*₁, ..., through *arg*_{*k*} is implemented in PLASMA by passing to *f* a request envelope with a message consisting of the tuple [*arg*₁, ..., *arg*_{*k*}] of arguments and a continuation actor to which the value of *f* should be sent.

Carl Hewitt. 1977. Viewing control structures as patterns of passing messages.

demo/reply.exs

Correlation Tokens

https://hexdocs.pm/elixir/Kernel.html#make_ref/0

```
make_ref()
```

Returns an almost unique reference.

demo/correlation.exs

demo/chat_client.exe

demo/chat_server.exs

Let it crash

The Erlang philosophy for handling errors can be expressed in a number of slogans:

- Let some other process do the error recovery.
- If you can't do what you want to do, die.
- Let it crash.
- Do not program defensively.

Joe Armstrong. 2003. Making reliable distributed systems in the presence of software errors.

Supervisors in Elixir

https://hexdocs.pm/elixir/Kernel.html#spawn_link/1

```
spawn_link(fun)
```

Spawns the given function, links it to the current process, and returns its PID.

<https://hexdocs.pm/elixir/Process.html#flag/2>

```
flag(:trap_exit, boolean())
```

Sets the given flag to value for the calling process.

<https://hexdocs.pm/elixir/Process.html#exit/2>

```
exit(pid, reason)
```

The following behavior applies if reason is any term except `:normal` or `:kill`:

1. If pid is not trapping exits, pid will exit with the given reason.
2. If pid is trapping exits, the exit signal is transformed into a message `{:EXIT, from, reason}` and delivered to the message queue of pid.

demo/supervisor.exs

demo/division.exs

Deadlocks

Deadlock, in a strict syntactic sense, can not exist in an actor system. In a higher level semantic sense of the term, deadlock can occur in a system of actors.

Gul Agha. 1986. Actors: a model of concurrent computation in distributed systems.

Dining Philosophers:

- Forks are actors
- Philosophers are actors

demo/dining.exs

demo/timeout.exs

Location Transparency

Generally, we want to refer to a resource by a high-level name, independent of its low-level location.

Examples:

- Phone contacts
- Support hotline
- Virtual memory
- File names
- Domain names

Even more important in distributed systems.

Location Transparency in Elixir

<https://hexdocs.pm/elixir/Process.html#register/2>

```
register(pid_or_port, name)
```

Registers the given `pid_or_port` under the given name on the local node.

<https://hexdocs.pm/elixir/Process.html#whereis/1>

```
whereis(name)
```

Returns the PID or port identifier registered under `name` or `nil` if the name is not registered.

demo/registry.exs

Nodes

```
iex --sname node1
```

<https://hexdocs.pm/elixir/Kernel.html#node/0>

```
node()
```

Returns an atom representing the name of the local node.

<https://hexdocs.pm/elixir/Node.html#connect/1>

```
Node.connect(node)
```

Establishes a connection to node.

demo/distributed.exs

Global Names

https://www.erlang.org/doc/apps/kernel/global.html#register_name/2

```
:global.register_name(Name, Pid)
```

Globally associates name Name with a pid, that is, globally notifies all nodes of a new global name in a network of Erlang nodes.

https://www.erlang.org/doc/apps/kernel/global.html#whereis_name/1

```
:global.whereis_name(Name)
```

Returns the pid with the globally registered name Name. Returns undefined if the name is not globally registered.

demo/global.exs

Fairness Guarantee in Elixir

The abstract machine uses reduction-based preemptive scheduling.

Each function call, message send, etc. costs 1 reduction.

The scheduler preempts a process after 2000 steps.

No starvation of processes is possible.

demo/fairness.exs

Summary and Outlook

Actors provide a natural structure for concurrent applications.

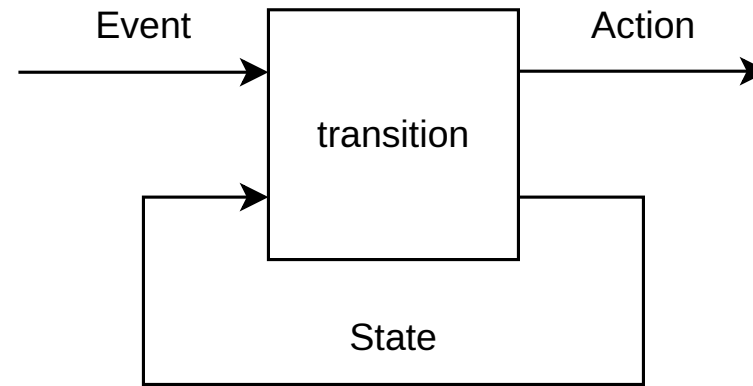
Next week: Model View Update

Model View Update

Feedback Loop

A mealy machine consists of:

- a set of states,
- a set of events,
- a set of actions,
- a start state,
- a transition function.



When to use it

- User interfaces
- Embedded systems
- Cyber-physical systems
- Computer games

```
while (1) { switch(event) { ... } }
```

Advantages:

- No deadlocks
- No races
- Testability
- Provability
- Resource bounds

Elm

<https://elm-lang.org/>

```
import Html exposing (text)

main =
  text "Hello!"
```

Designed by Evan Czaplicki, first appeared 2013.

Elm compiles to JavaScript.

Totality

Total functions are defined for every value in their domain.

Partial functions need not be defined for every value in their domain.

In programming partiality is modeled with exceptions and panics.

Elm is a total functional programming language: there are no exceptions.

(Disregarding non-termination)

demo/Hello.elm

Feedback Loop in Elm

```
type Model  
  
init : Model  
  
update : Event -> Model -> Model  
  
view : Model -> Html Event
```

This idea has various names:

- The Elm Architecture
- Reactor pattern
- Event-driven architecture
- Universe pattern (c.f. Praktische Informatik 1)
- Unidirectional data flow

demo/Increment.elm

Events in Elm

<https://package.elm-lang.org/packages/elm/html/1.0.1/Html>

```
text : String -> Html msg
button : List (Attribute msg) -> List (Html msg) -> Html msg
input : List (Attribute msg) -> List (Html msg) -> Html msg
```

<https://package.elm-lang.org/packages/elm/html/1.0.1/Html-Events>

```
onClick : msg -> Attribute msg
onInput : (String -> msg) -> Attribute msg
```

demo/Payload.elm

Actions in Elm

<https://package.elm-lang.org/packages/elm/core/latest/Platform-Cmd#Cmd>

```
type Cmd msg
```

A command is a way of telling Elm, "Hey, I want you to do this thing!" [...]

Every `Cmd` specifies (1) which effects you need access to and (2) the type of messages that will come back into your application.

```
type Model  
  
init : () -> ( Model, Cmd Event )  
  
update : Event -> Model -> ( Model, Cmd Event )  
  
view : Model -> Html Event
```

Tasks in Elm

<https://package.elm-lang.org/packages/elm/core/latest/Task#perform>

```
perform : (a -> msg) -> Task Never a -> Cmd msg
```

Like I was saying in the Task documentation, just having a Task does not mean it is done.

<https://package.elm-lang.org/packages/elm/core/latest/Process#sleep>

```
sleep : Float -> Task Never ()
```

Block progress on the current process for the given number of milliseconds.

demo/Sleep.elm

Fetch in Elm

<https://package.elm-lang.org/packages/elm/http/latest/Http#get>

```
get : { url : String, expect : Expect msg } -> Cmd msg
```

Create a GET request.

<https://package.elm-lang.org/packages/elm/http/latest/Http#expectString>

```
expectString : (Result Error String -> msg) -> Expect msg
```

Expect the response body to be a `String`.

demo/Fetch.elm

Subscriptions in Elm

```
subscriptions : Model -> Sub Event
```

<https://package.elm-lang.org/packages/elm/core/latest/Platform.Sub>

```
type Sub msg
```

A subscription is a way of telling Elm, "Hey, let me know if anything interesting happens over there!" [...] Every Sub specifies (1) which effects you need access to and (2) the type of messages that will come back into your application.

<https://package.elm-lang.org/packages/elm/time/latest/Time#every>

```
every : Float -> (Posix -> msg) -> Sub msg
```

Get the current time periodically.

demo/Ticker.elm

Composition

We want to compose models, views, and updates of different components.

Product type of models:

```
type alias Model = { model1 : Model1, model2 : Model2 }
```

Sum type of events:

```
type Event = Event1 Event1 | Event2 Event2
```

Event routing in update:

```
update event model = case event of  
  Event1 event1 -> ... update1 event1 model.model1 ...  
  Event2 event2 -> ... update2 event2 model.model2 ...
```

Visual composition in view:

```
view model =  
  ... view1 model.model1 ... view2 model.model2 ...
```

demo/Composition.elm

Virtual Document Object Model

The browser maintains a tree of visual components: the document object model.

Each change to this tree is expensive.

To minimize and batch changes, we keep a virtual copy of this tree.

We calculate the difference to this virtual tree and only apply those changes.

demo/Virtual.elm

Summary and Outlook

Model View Update is simple but verbose.

Next week: Remote Procedures

Remote Procedures

Distributed Systems

A distributed system consists of a collection of distinct processes which are spatially separated, and which communicate with one another by exchanging messages. A network of interconnected computers, such as the ARPA net, is a distributed system.

Leslie Lamport. 1978. Time, clocks, and the ordering of events in a distributed system.

Defining characteristic: crashing processes and unreliable communication.

All problems of concurrency and more.

A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.

Attributed to Leslie Lamport. 1987.

Everything fails all the time

Werner Vogels.

Avoid distributed systems!

Failure

Millions of websites offline after fire at French cloud services firm

By Reuters, Matthieu Rosemain and Raphael Satter

March 10, 2021 8:44 AM GMT+1 · Updated 4 years ago



[U4] General view of firefighters working to extinguish a fire at the French cloud computing company OVHcloud in Strasbourg, France, March 10, 2021. Separators pompiers du feu (Remy/Handout via REUTERS) Purchase Licensing Rights

PARIS, March 10 (Reuters) - A fire at a French cloud services firm has disrupted millions of websites, knocking out government agencies' portals, banks, shops, news websites and taking out a chunk of the .FR web space, according to internet monitors.

<https://reuters.com/world/blaze-destroys-servers-europes-largest-cloud-services-firm-2021-03-10/>

Amazon reveals cause of AWS outage that took everything from banks to smart beds offline

AWS explains in a lengthy post how a bug in automation software brought down thousands of sites and applications



Signal, Snapchat, Reddit, Duolingo and Ring doorbells were some of the 2,000 companies affected by this week's AWS outage, according to Downdetector. Photograph: Anushree Fadnis/Reuters

Amazon has revealed the cause of this week's **hours-long AWS outage**, which took everything from Signal to smart beds offline, was a bug in automation

<https://www.theguardian.com/technology/2025/oct/24/amazon-reveals-cause-of-aws-outage>

<https://blog.cloudflare.com/18-november-2025-outage/>

Faults and Failures

Fault: root cause (e.g., hardware dies, software bug, cable broke)

Error: inconsistent internal state (e.g., missing values, index out of bounds)

Failure: system deviates from specification (e.g., unresponsive, wrong)

Machine Faults

Partial fault: some component crashes.

Crash-stop:

- a disk is damaged
- one server crashes
- permanent

Crash-reboot:

- power outage
- network partition
- transient

Byzantine:

- arbitrary, even malicious behavior

Should not lead to system failure!

Network Faults

Messages might be:

- delayed
- omitted
- duplicated
- reordered
- corrupted
- malicious

Should not lead to system failure!

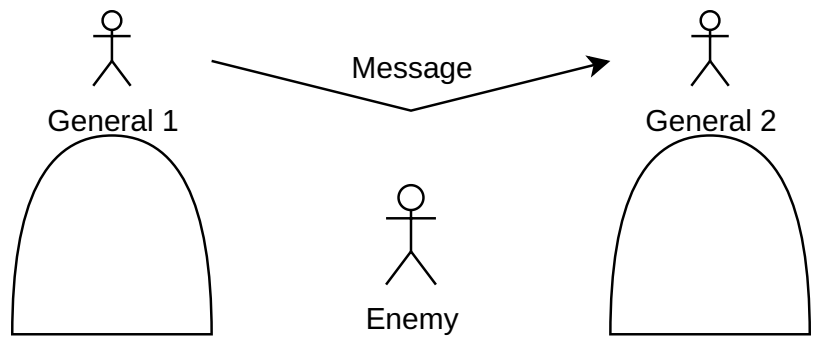
Perfect fault detection is impossible. Can't distinguish between slow and dead.

Two Generals

Two generals must coordinate their attack.

If both attack they win. If only one attacks they both lose.

But messages may get lost.

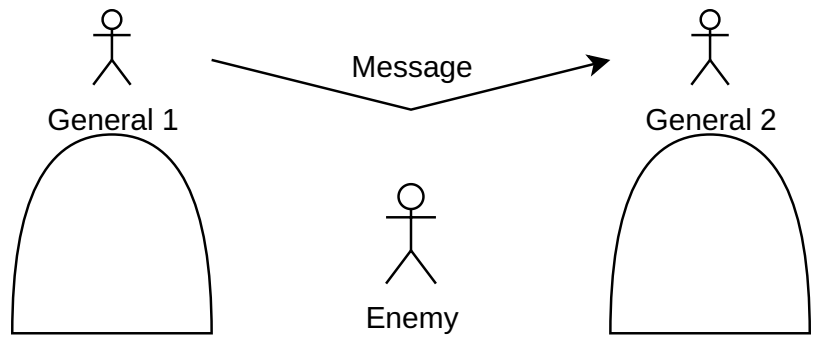


Two Generals

Two generals must coordinate their attack.

If both attack they win. If only one attacks they both lose.

But messages may get lost.



Theoretical problem: perfect agreement over an unreliable channel is impossible.

Practical solution: assume messages eventually arrive after a finite number of retries.

When to use them

Physically distributed

Big data

Big compute

Fault tolerance

Legal requirements

Only when necessary!

Remote Procedure Calls

Make distributed programming look like normal programming.

Transfer of control and data from one machine to another one and back.

A principle that we used several times in making design choices is that the semantics of remote procedure calls should be as close as possible to those of local (single-machine) procedure calls.

Andrew D. Birrell and Bruce Jay Nelson. 1984. Implementing remote procedure calls.

gRPC

<https://grpc.io>

```
with grpc.insecure_channel("localhost:50051") as channel:  
    stub = helloworld_pb2_grpc.GreeterStub(channel)  
    response = stub.SayHello(helloworld_pb2.HelloRequest(name="you"))  
print("Greeter client received: " + response.message)
```

Created by Google, first released 2015.

demo/helloworld/python/main.py

Interface Definition Language

Define data structures and function signatures once.

Generate compatible code for multiple languages.

Protobuf

```
service Greeter {  
  rpc SayHello (HelloRequest) returns (HelloReply) {}  
}
```

Created by Google, first released 2008.

<https://reasonablypolymorphic.com/blog/protos-are-wrong/>

demo/helloworld/greeter.proto

Stubs

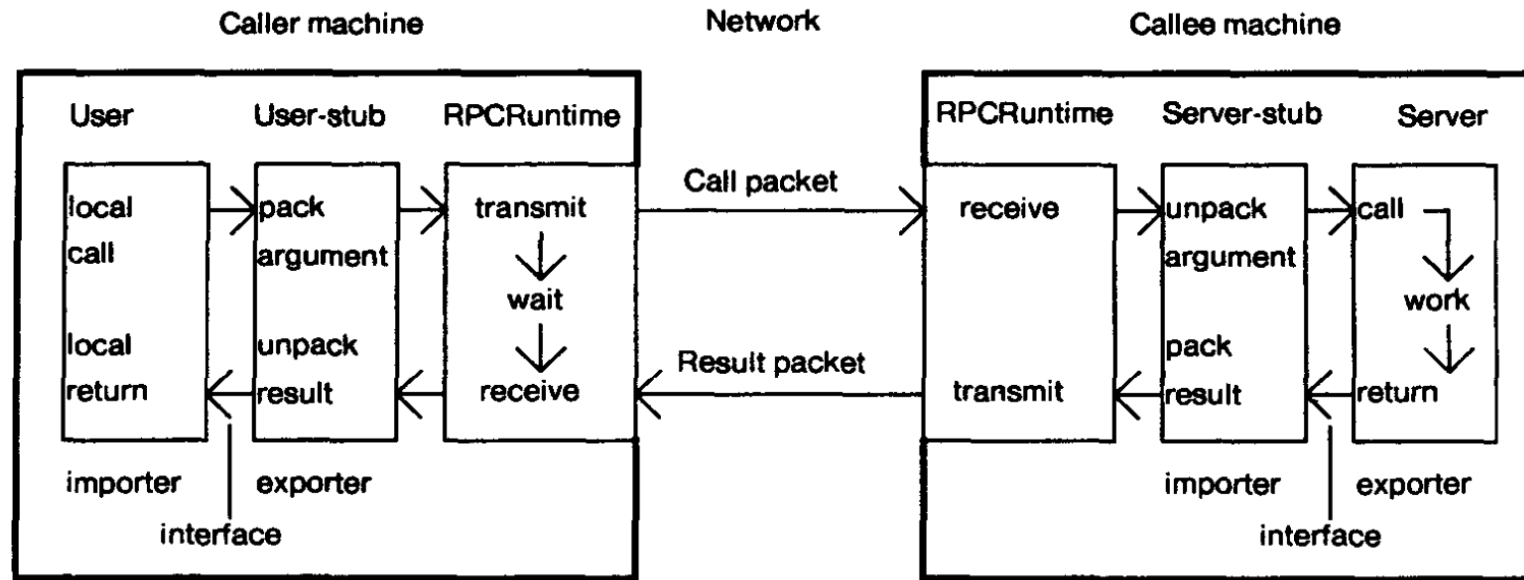


Fig. 1. The components of the system, and their interactions for a simple call.

Andrew D. Birrell and Bruce Jay Nelson. 1984. Implementing remote procedure calls.

demo/helloworld/python/greeter_pb2_grpc.py

demo/helloworld/go/greeter/greeter_grpc.pb.go

Serialization

- Sum types: tagged unions or an emulation.
- Recursive Types: flattened usually depth-first.
- Immutable references: integer identifiers of entities.
- Exceptions: caught and re-raised at the caller.
- Mutable references: do not make sense since address space is not shared.

demo/serialization/types.proto

Request-and-Response versus Fire-and-Forget

We guarantee that if the call returns to the user then the procedure in the server has been invoked precisely once. Otherwise, an exception is reported to the user and the procedure will have been invoked either once or not at all - the user is not told which.

Andrew D. Birrell and Bruce Jay Nelson. 1984. Implementing remote procedure calls.

Downsides of fire-and-forget:

- no confirmation
- no backpressure

Downsides of request-and-response:

- waiting
- coupling

Timeouts

Provided the RPCRuntime on the server machine is, still responding, there is no upper bound on how long we will wait for results; that is, we will abort a call if there is a communication breakdown or a crash but not if the server code deadlocks or loops. This is identical to the semantics of local procedure calls.

Andrew D. Birrell and Bruce Jay Nelson. 1984. Implementing remote procedure calls.

Network-level timeout:

- how long we wait for the network
- based on Round Trip Time (RTT)

Application-level timeout:

- how long we wait for the response
- based on Service Level Agreement (SLA)

In theory, no application-level timeout is needed. In practice, you should set one!

demo/helloworld/python/timeout.py

Retries

In addition to exceptions raised by the callee, the RPCRuntime may raise a call failed exception if there is some communication difficulty. This is the primary way in which our clients note the difference between local and remote calls.

Andrew D. Birrell and Bruce Jay Nelson. 1984. Implementing remote procedure calls.

Network errors (unavailable, timeout): retry!

Other errors (invalid argument, version mismatch, key not found): do not retry!

Problems with Retries

Thundering Herd: synchronized retries

- Solution: jitter (randomly delay retry)

Retry Storm: retries overload already-failing server

- Solution: exponential backoff (wait longer and longer for retry)
- Solution: circuit breaker (stop retrying after threshold, resume later)

Retry Amplification: retries multiply through call chain

- Solution: retry budgets (limit percentage of requests that are retries)
- Solution: limited depth (don't retry if you already are in a retry)

demo/helloworld/python/retries.py

Summary and Outlook

Remote procedures are simple and useful.

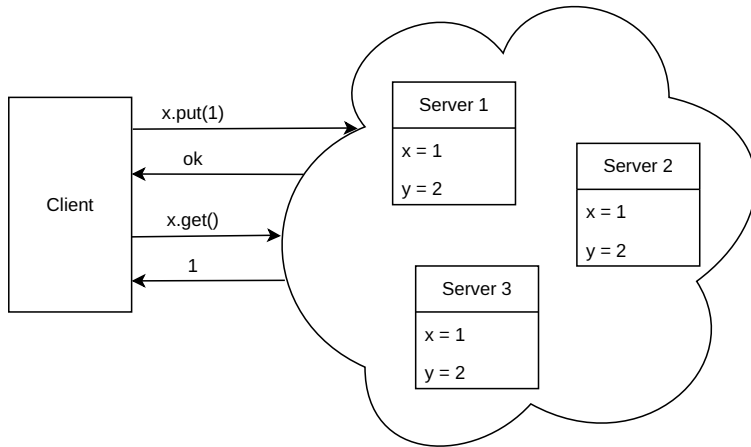
Next week: Replicated Data

Replicated Data

Distributed Data

A distributed system is a collection of independent computers that appears to its users as a single coherent system.

Andrew S. Tanenbaum and Maarten van Steen. 2006. Distributed Systems: Principles and Paradigms (2nd Edition).



When to use it

Reason 1: reliability, i.e., fault tolerance.

Reason 2: performance, i.e., very large data.

With retries we execute operations at most once.

Now we need to execute operations exactly once.

Later we will execute operations at least once.

Examples: distributed databases, key-value stores, collaborative editors, blockchains, ...

State Machine Replication

Fred B. Schneider. 1990. Implementing fault-tolerant services using the state machine approach: a tutorial.

Mealy machine: states, operations, outputs, deterministic transition.

Goal: all replicas execute the same operations in the same order.

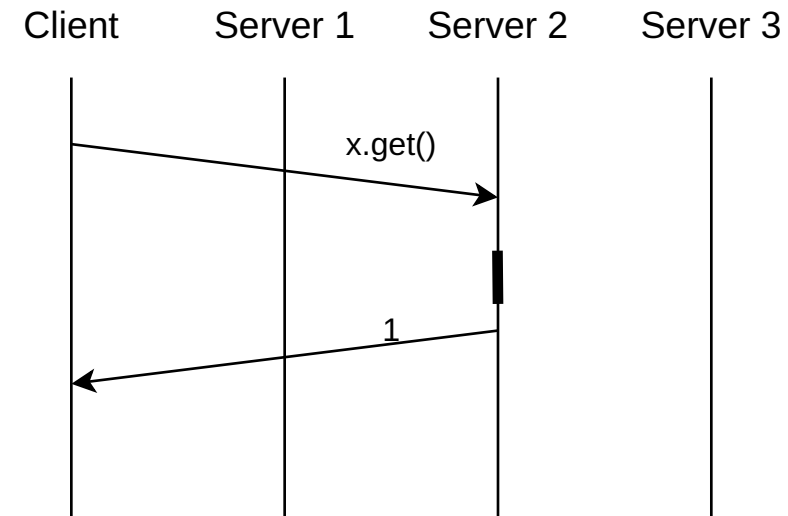
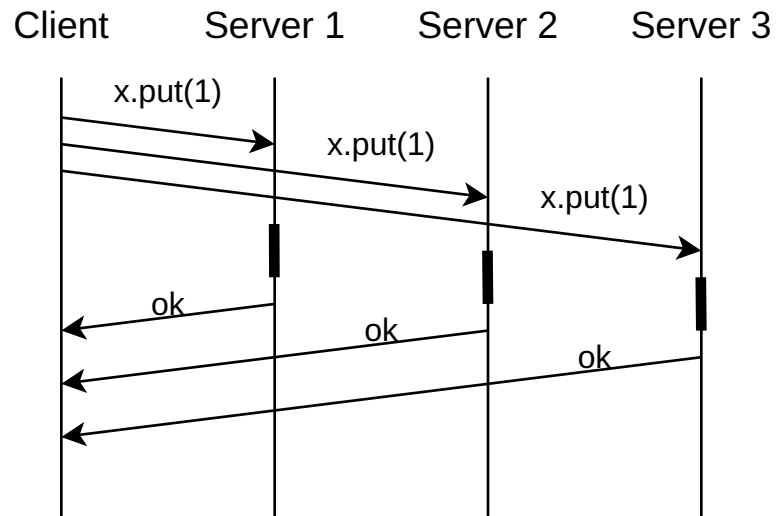
Reliability: every live process reliably receives every operation.

Ordering: every live process applies them in the same order.

Running example: a key-value store with operations `put` and `get` .

Active Replication

Client sends puts to all replicas and waits for acknowledgment and sends gets to any replica.



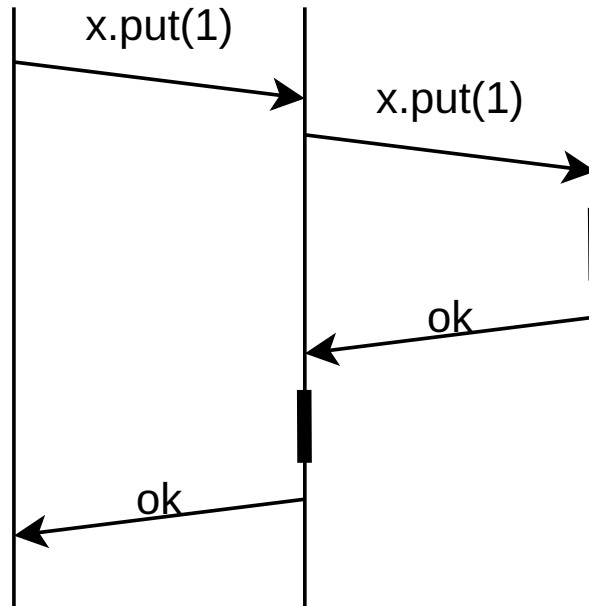
Put: write and acknowledge.

Get: read and respond.

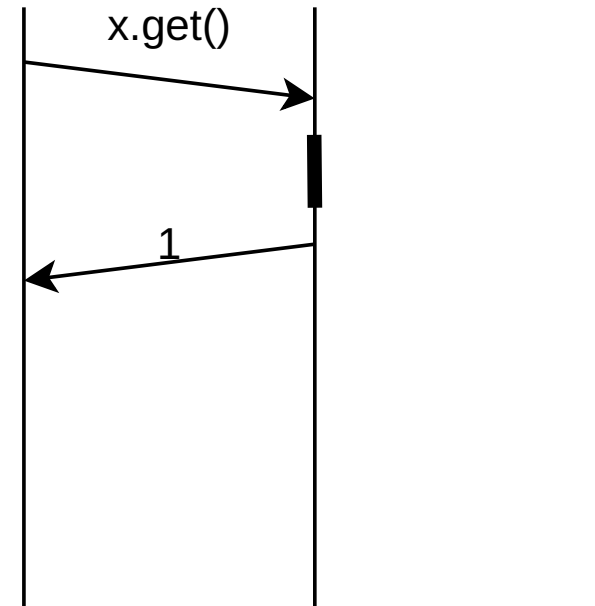
Primary-Backup Replication

Client sends puts and gets to primary.

Client Primary Backup



Client Primary Backup

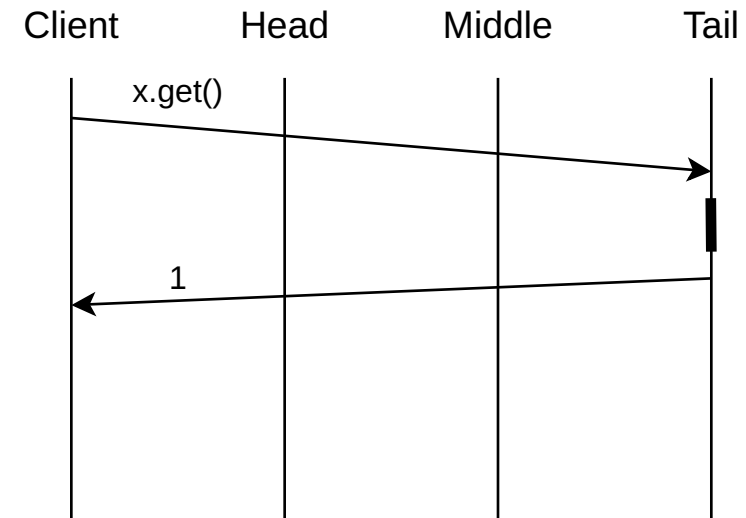
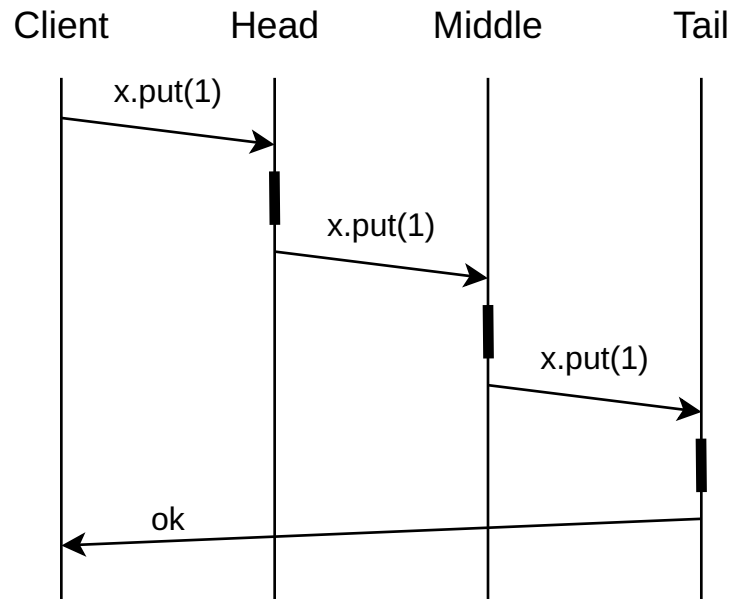


Put: write and wait for acknowledgment from backup.

Get: read and respond.

Chain Replication

Client sends puts to head and gets to tail.



Put: write and send on.

Get: read and respond.

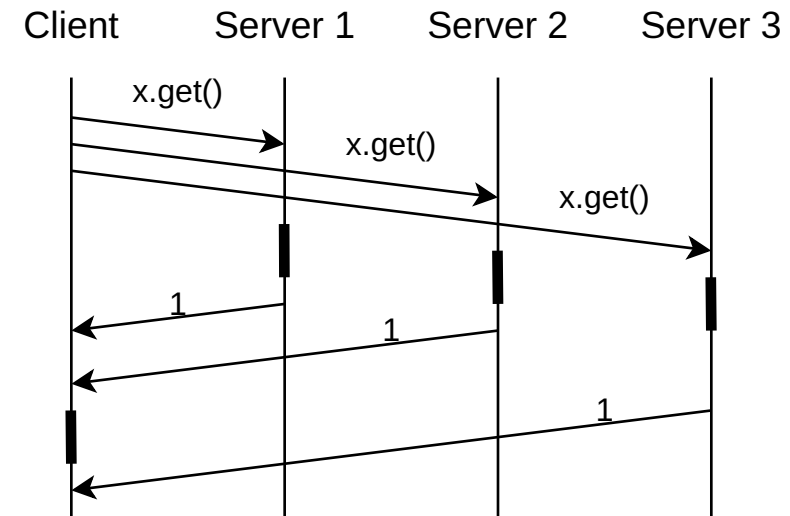
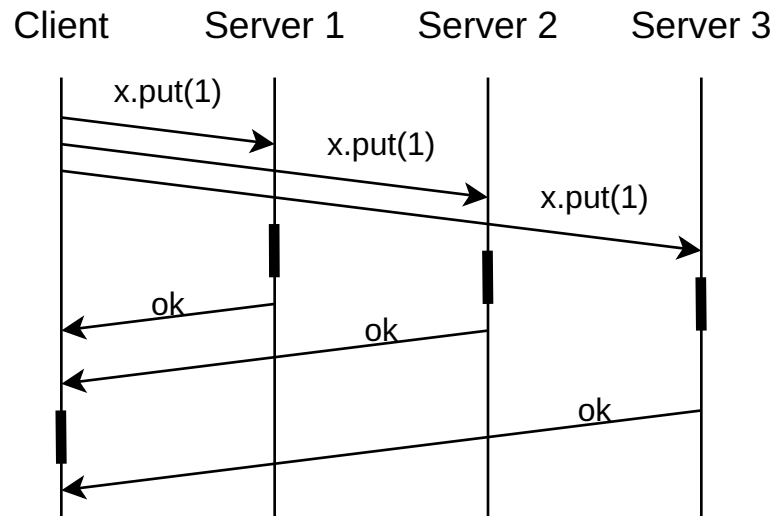
Quorum Protocols

Client waits for enough acknowledgments. Tunable: $R + W > N$ versus not.

N: number of replicas

W: votes required for write

R: votes required for read

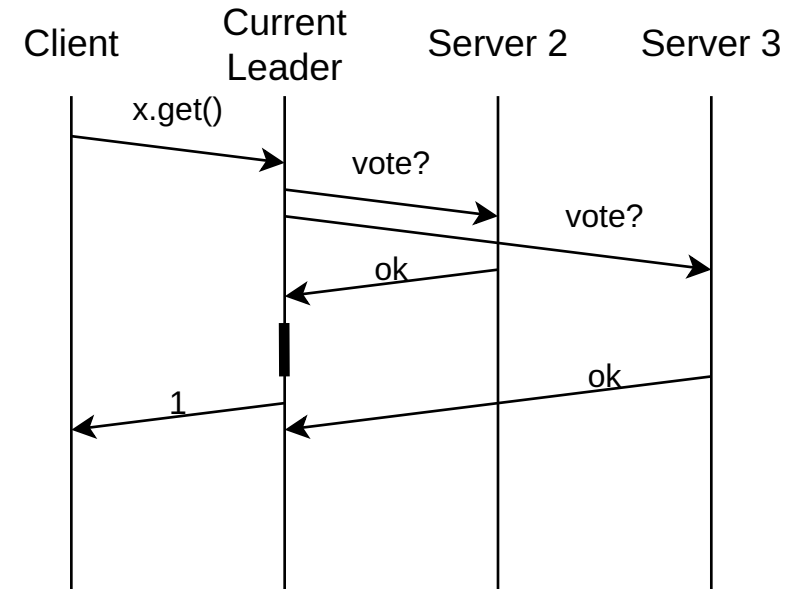
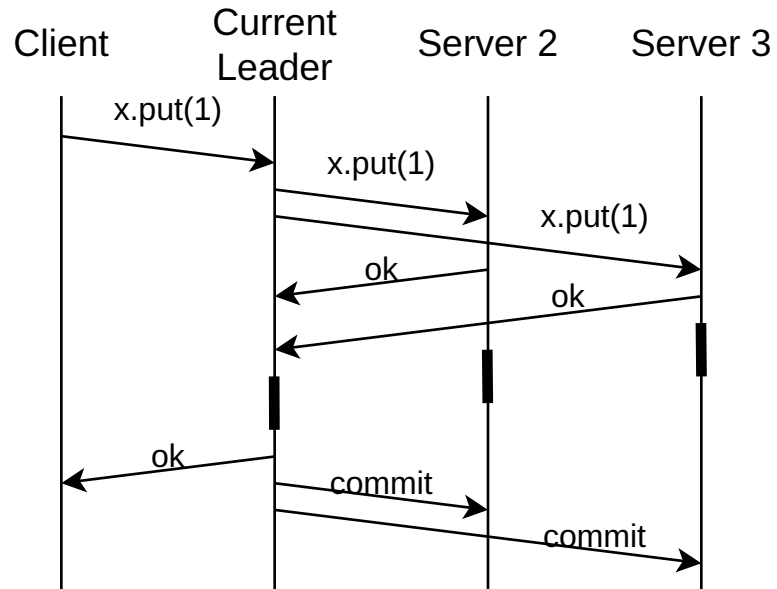


Put: write and acknowledge.

Get: read and respond.

Consensus Protocols

Client sends operation to any server.



Very complicated.

Paxos: Leslie Lamport. 1998. The part-time parliament.

Raft: Diego Ongaro and John Ousterhout. 2014. In search of an understandable consensus algorithm.

Machine Crashes

Active replication: tolerates 0 crashes for writes (N - 1) crashes for reads.

Primary backup replication: tolerates 1 crash with backup taking over.

Chain replication: tolerates (N - 1) crashes with bypassing in chain.

Quorum replication: tolerates (N - W) crashes for writes and (N - R) crashes for reads.

Consensus protocol: tolerates F failures when $N = 2 * F + 1$

Concurrent Operations

Only possible with multiple clients.

Active replication: replicas receive and execute operations in different order.

Primary backup replication: all clients talk to same server which orders operations.

Chain replication: clients talk to the current head which orders operations.

Quorum replication: replicas receive and execute operations in different order.

Consensus protocol: clients talk to any server but current leader decides order.

Ordering Schemes

etcd (CoreOS): consensus-based total order via Raft

Diego Ongaro and John Ousterhout. 2014. In search of an understandable consensus algorithm.

Dynamo (Amazon): version vectors and reconciliation

Giuseppe DeCandia, et al. 2007. Dynamo: amazon's highly available key-value store.

Spanner (Google): real time with global positioning system and atomic clocks

James C. Corbett, et al. 2013. Spanner: Google's Globally Distributed Database.

Calvin (Yale): sequencer processes

Alexander Thomson, et al. 2012. Calvin: fast distributed transactions for partitioned database systems.

Performance

Read Latency

Active replication: low

Primary backup replication: low

Chain replication: low

Quorum replication: tunable

Consensus protocol: medium

Write Latency

Active replication: low

Primary backup replication: medium

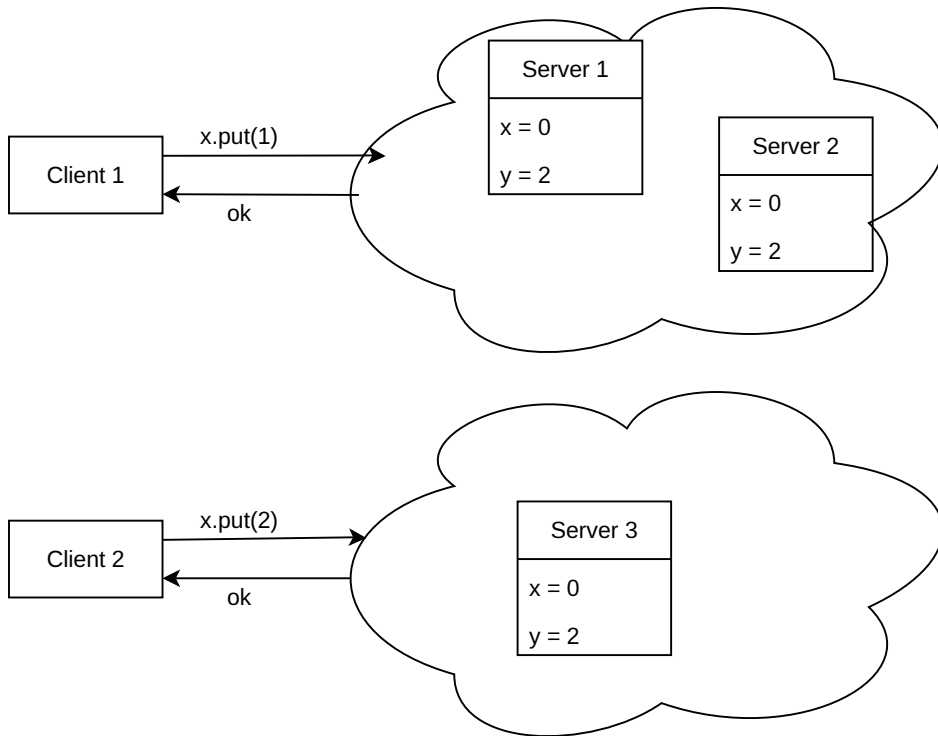
Chain replication: high

Quorum replication: tunable

Consensus protocol: high

Network Partitions

The network splits into multiple parts that can not communicate anymore.



CAP theorem: during the partition, choose between consistency and availability.

Consistency Levels

Linearizability: Every operation appears to execute atomically at some point between invocation and response, respecting real-time order.

Sequential Consistency: All operations appear in the same total order to all processes, preserving each process's program order.

Causal Consistency: Operations that are related via happened-before are seen in the same order by all processes, concurrent operations may be seen differently.

Consistency

Active replication: causal consistency by using version vectors.

Primary backup replication: linearizable by single sequencer.

Chain replication: linearizable by single sequencer.

Quorum replication: causal consistency by using version vectors.

Consensus protocol: linearizable by using lots of things.

Temporal Logic

Propositional logic extended with two modalities: $\Box$ (always) and $\Diamond$ (eventually).

Axioms:

$$\Box(P \rightarrow Q) \rightarrow (\Box P \rightarrow \Box Q)$$

$$\Box P \rightarrow P$$

$$\Box P \rightarrow \Box \Box P$$

$$\Diamond(P \rightarrow Q) \rightarrow (\Diamond P \rightarrow \Diamond Q)$$

$$P \rightarrow \Diamond P$$

$$\Diamond \Diamond P \rightarrow \Diamond P$$

Safety properties: bad things never happen.

Liveness properties: good things eventually happen.

<https://lamport.azurewebsites.net/tla/science-book.html>

TLA+

<https://lamport.azurewebsites.net/tla/tla.html>

```
----- MODULE HourClock -----  
EXTENDS Naturals  
  
VARIABLES hour  
  
Init == hour = 1  
  
Next == hour' = IF hour = 12 THEN 1 ELSE hour + 1  
  
Spec == Init /\ [][Next]_hour  
=====
```

Designed by Leslie Lamport, first appeared 1999.

Quint

<https://quint-lang.org>

```
var balances: str -> int

pure val ADDRESSES = Set("alice", "bob", "charlie")

action withdraw(account, amount) = {
  balances' = balances.setBy(account, curr => curr - amount)
}

val no_negatives = ADDRESSES.forall(addr =>
  balances.get(addr) >= 0
)
```

Designed by the Informal Systems team, first released 2022.

Modern syntax and tooling for TLA+.

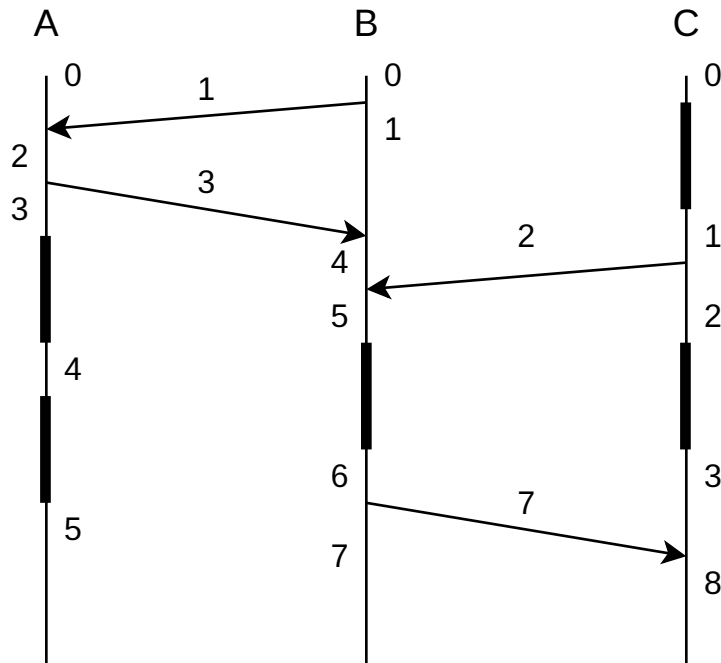
demo/basics.qnt

demo/network.qnt

Logical Time

The happened-before relation is only a partial order.

A logical clock provides a total order that respects this partial order.



Two implementation rules:

- IR1. Each process P_i increments C_i between any two successive events.
- IR2. (a) If event a is the sending of a message m by process P_i , then the message m contains a timestamp $T_m = C_i(a)$.
(b) Upon receiving a message m , process P_i sets C_i greater than or equal to its present value and greater than T_m .

Use process names to break ties.

Vector Clocks

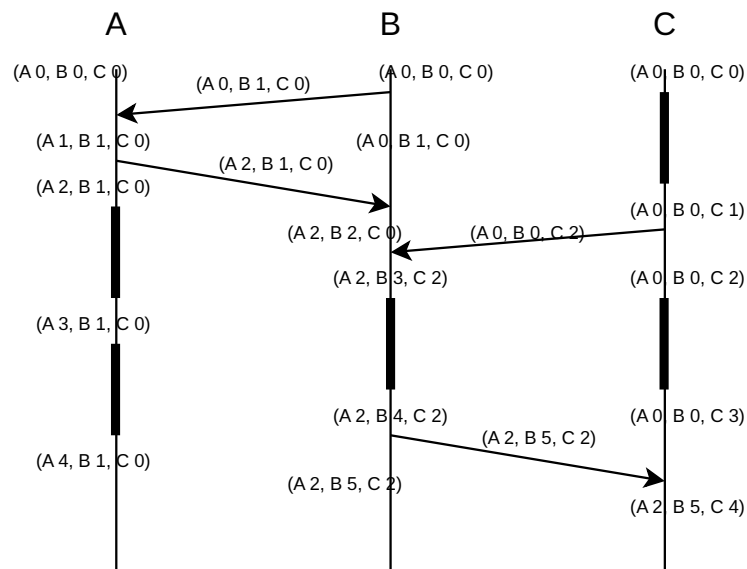
Colin J. Fidge. 1988. Timestamps in message-passing systems that preserve the partial order.

Friedemann Mattern. 1989. Virtual time and global states of distributed systems.

Vector clocks form a partial order that is precisely happened-before.

Makes it possible to detect concurrent events.

Each process maintains a vector of counters, one entry for each process.



Each process increments its own entry as with Lamport clocks.

Processes attach the vector when sending messages.

Processes update the vector with the pointwise maximum when receiving messages.

demo/causal.qnt

Local-first Data

However, by centralizing data storage on servers, cloud apps also take away ownership and agency from users. If a service shuts down, the software stops functioning, and data created with that software is lost.

In this article we propose “local-first software”: a set of principles for software that enables both collaboration *and* ownership for users. Local-first ideals include the ability to work offline and collaborate across multiple devices, while also improving the security, privacy, long-term preservation, and user control of data.

Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. 2019. Local-first software: you own your data, in spite of the cloud.

<https://www.inkandswitch.com/essay/local-first/>

Coordination-free Computation

Coordination protocols enable autonomous, loosely coupled machines to jointly decide how to control basic behaviors, including the order of access to shared memory. These protocols are among the most clever and widely cited ideas in distributed computing.

The issue is not that coordination is tricky to implement, though that is true. The main problem is that coordination can dramatically slow down computation or stop it altogether.

Joseph Hellerstein and Peter Alvaro. 2020. Keeping CALM: when distributed consistency is easy.

<https://cacm.acm.org/research/keeping-calm/>

Semilattices

Associativity: $(a + b) + c = a + (b + c)$ (for composition).

Commutativity: $a + b = b + a$ (for permutation).

Idempotence: $a + a = a$ (for duplication).

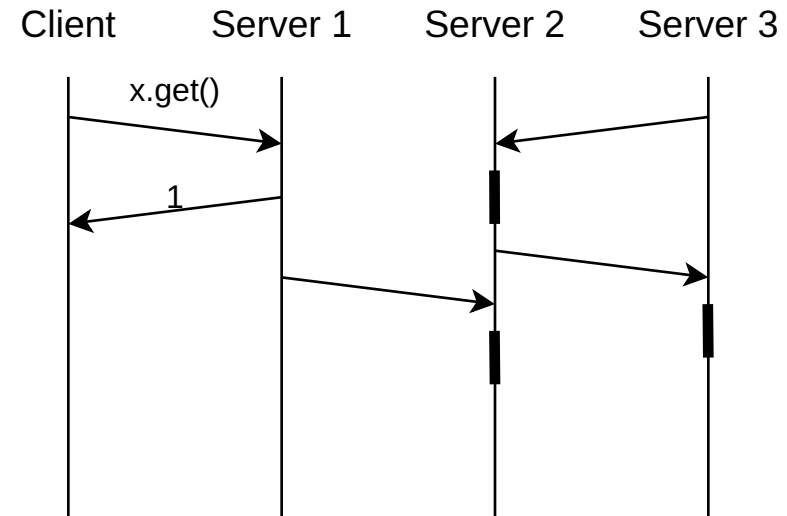
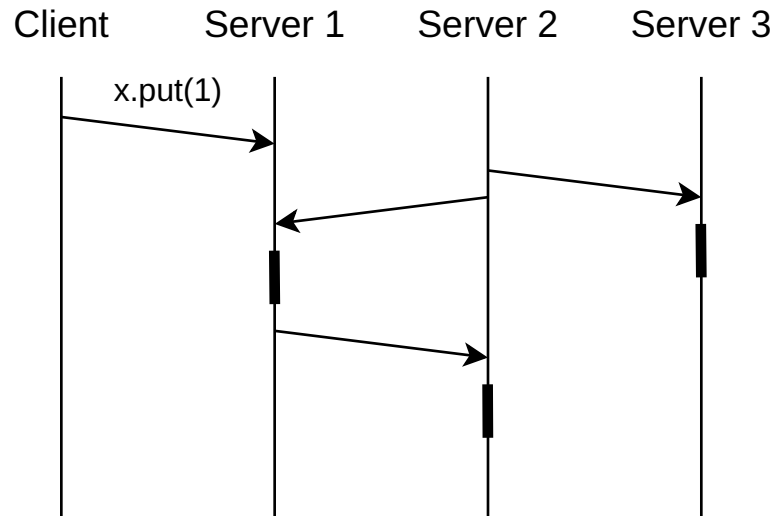
These properties induce an order: $a \leq b$ is defined as $a + b = b$.

Inflation: $a \leq f(a)$ (for operations).

Monotonicity: when $a \leq b$ then $f(a) \leq f(b)$ (not needed).

Gossip Protocol

Client sends operation to any server.



Servers exchange and merge their states independent of clients.

Conflict-free Replicated Datatypes (CRDTs)

Paulo Sérgio Almeida. 2024. Approaches to Conflict-free Replicated Data Types.

```
type S  
merge : (S × S) -> S  
update : U -> (S -> S)  
view : S -> V
```

Where `merge` forms a semilattice and `update u` is inflationary.

Properties of Gossip Protocol with CRDT

Network Partitions: no problem.

Machine Crashes: no problem.

Read Latency: low

Write Latency: low

Eventual Consistency: If no new updates are made, all reads eventually return the same value.

When to use it

Mobile applications

Collaborative editing

Edge computing

Geo-distributed databases

Examples:

- Automerge: <https://automerge.org/>
- Yjs: <https://yjs.dev/>

demo/counter.qnt

Grow-only Counter

A counter with an operation `increment` .

```
type S = Vector[Nat]
merge s1 s2 = zipWith(max, s1, s2)
increment s = s[i] += 1
view s = sum(s)
```

Where `i` is index of executing replica.

Positive-Negative Counter

A counter with operations `increment` and `decrement`.

```
type S = (Vector[Nat], Vector[Nat])

merge (p1, n1) (p2, n2) = (zipWith(max, p1, p2), zipWith(max, n1, n2))

increment (p, n) = (p[i] += 1, n)
decrement (p, n) = (p, n[i] += 1)

view (p, n) = sum(p) - sum(n)
```

Where `i` is index of executing replica.

Observed-Reset Counter

A counter with operations `increment` and `reset` .

A replica only resets those increments it has observed.

```
type S = (Vector[Nat], Vector[Nat])

merge (p1, n1) (p2, n2) = (zipWith(p1, p2, max), zipWith(n1, n2, max))

increment (p, n) = (p[i] += 1, n)
reset (p, n) = (p, zipWith(max, p, n))

view (p, n) = sum(p) - sum(n)
```

Where `i` is index of executing replica.

Grow-only Set

A set with an operation `add` .

```
type S[A] = Set[A]
merge s1 s2 = union(s1, s2)
add s x = insert(s, x)
view s = s
```

Elements, once inserted, can never be removed.

Two-Phase Set

A set with operations `add` and `remove`.

```
type S[A] = (Set[A], Set[A])  
  
merge (a1, r1) (a2, r2) = (union(a1, a2), union(r1, r2))  
  
add (a, r) x = (insert(a, x), r)  
  
remove (a, r) x = (a, insert(r, x))  
  
view (a, r) = difference(a, r)
```

Elements, once inserted and removed, can never be inserted again.

We keep a set of tombstones.

Multi-value Register

A register with an operation `put`.

Reading from the register might result in multiple values.

```
type Dot = (ID, Nat)
type S[T] = (Set[(T, Dot, Set[Dot])], Set[Dot])

merge (w1, o1) (w2, o2) = (union(w1, w2), union(o1, o2))

put (w, o) v = (insert(w, (v, (i, n), o)), insert(o, (i, n)))
  where `i` is index of executing replica
  where `n` is the next fresh version at this replica

view (w, o) = filter overwritten writes from w
```

Observed writes are overwritten, concurrent writes are kept.

Text

The key idea of most text CRDTs is to represent the text as a sequence of characters, and to give each character a unique identifier.

To insert a character into a document, we generate an insert operation of the following form:

```
{ action: "insert", opId: "2@alice", afterId: "1@alice", character: "x" }
```

This operation inserts the character “x” after the existing character whose ID is 1@alice.

To delete a character from a document, we generate a remove operation of the following form:

```
{ action: "remove", opId: "5@alice", removedId: "2@alice" }
```

This operation removes the existing character whose ID is 2@alice (i.e. the “x” we inserted above).

Summary and Outlook

Replicating data and keeping it consistent is hard.

Next week: Collective Computation.

Collective Computation

Bulk Synchronous Parallel (BSP)

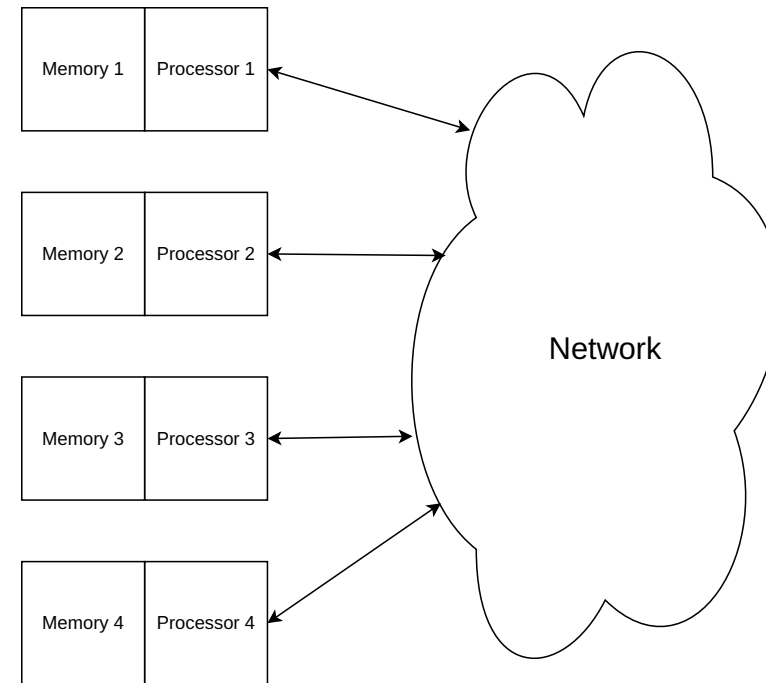
Due to the rapidly decreasing cost of processing, memory, and communication, it has appeared inevitable for at least two decades that parallel machines will eventually displace sequential ones in computationally intensive domains.

Leslie G. Valiant. 1990. A bridging model for parallel computation.

Multiple processors with local memory.

Any processor can communicate with any other processor through the network.

Minimize communication!



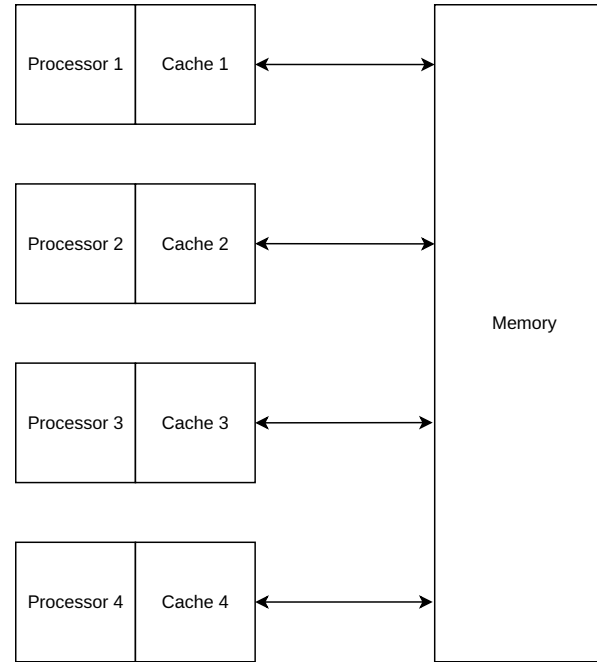
Not Distributed Shared Memory

Parallel Random Access Machine (PRAM)

Fine-grained writes to shared state

Does not scale:

- cache coherence makes it slow
- false sharing makes it slow
- fault tolerance makes it slow



Steven Fortune and James Wyllie. 1978. Parallelism in random access machines.

Distributed Tasks

A Dryad application developer can specify an arbitrary directed acyclic graph to describe the application's communication patterns, and express the data transport mechanisms (files, TCP pipes, and shared-memory FIFOs) between the computation vertices.

Michael Isard, et al. 2007. Dryad: distributed data-parallel programs from sequential building blocks.

Vertices are connected by channels, producing and consuming streams.

Matei Zaharia, et al. 2012. Resilient distributed datasets: a fault-tolerant abstraction for in-memory cluster computing.

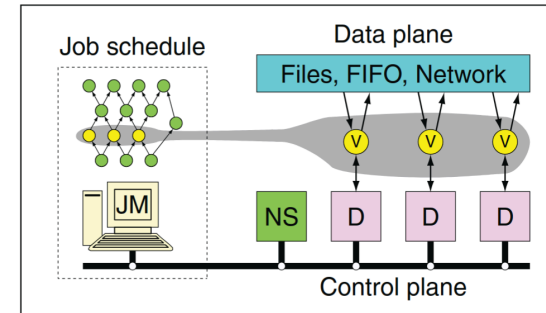


Figure 1: The Dryad system organization. The job manager (JM) consults the name server (NS) to discover the list of available computers. It maintains the job graph and schedules running vertices (V) as computers become available using the daemon (D) as a proxy. Vertices exchange data through files, TCP pipes, or shared-memory channels. The shaded bar indicates the vertices in the job that are currently running.

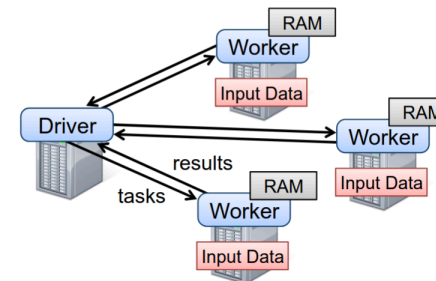


Figure 2: Spark runtime. The user’s driver program launches multiple workers, which read data blocks from a distributed file system and can persist computed RDD partitions in memory.

Spark

<https://spark.apache.org/>

```
>>> textFile.filter(textFile.value.contains("Spark")).count()  
15
```

We show that Spark is up to 20× faster than Hadoop for iterative applications, speeds up a real-world data analytics report by 40×, and can be used interactively to scan a 1 TB dataset with 5–7s latency.

Matei Zaharia, et al. 2012. Resilient distributed datasets: a fault-tolerant abstraction for in-memory cluster computing.

Scala

Compiled, statically-typed programming language for the JVM.

Functional, imperative, and object-oriented.

```
val fruits =  
  List("apple", "banana", "avocado", "papaya")  
  
val countsToFruits =  
  fruits.groupBy(fruit => fruit.count(_ == 'a'))  
  
for (count, fruits) <- countsToFruits do  
  println(s"with 'a' × $count = $fruits")
```

Created by Martin Odersky in 2004.

demo/Basics.scala

Resilient Distributed Dataset (RDD)

RDDs are fault-tolerant, parallel data structures that let users explicitly persist intermediate results in memory, control their partitioning to optimize data placement, and manipulate them using a rich set of operators.

Matei Zaharia, et al. 2012. Resilient distributed datasets: a fault-tolerant abstraction for in-memory cluster computing.

Transformations:

```
map(f: T => U): RDD[T] => RDD[U]
filter(f: T => Boolean): RDD[T] => RDD[T]
flatMap(f: T => Seq[U]): RDD[T] => RDD[U]
union(): (RDD[T], RDD[T]) => RDD[T]
```

Actions:

```
count(): RDD[T] => Long
collect(): RDD[T] => Seq[T]
reduce(f: (T, T) => T): RDD[T] => T
save(path: String) // Outputs to a storage system
```

demo/Laziness.scala

Partitioning

Distribution of data determines distribution of computation.

Hash partitioning: assign data by a hash modulo the number of processors.

Example with four processors:

```
("apple", 1) → hash("apple") % 4 = 2 → Processor 2  
("banana", 2) → hash("banana") % 4 = 1 → Processor 1  
("apple", 3) → hash("apple") % 4 = 2 → Processor 2  
("cherry", 4) → hash("cherry") % 4 = 0 → Processor 0
```

Related to horizontal sharding in databases.

Partitioning in Spark

Different partitioning strategies:

```
sc.parallelize[T](seq: Seq[T]): RDD[T] // split into ranges  
sc.textFile(path: String): RDD[String] // split into blocks  
repartition(numPartitions: Int): RDD[V] => RDD[V] // hash of value
```

For key value pairs partitioning is based on the key:

```
mapValues(f: V => W): RDD[(K, V)] => RDD[(K, W)] // preserves partitioning  
groupByKey(): RDD[(K, V)] => RDD[(K, Seq[V])] // hash of key  
reduceByKey(f: (V, V) => V): RDD[(K, V)] => RDD[(K, V)] // hash of key
```

demo/Grouping.scala

Map Reduce

Map, written by the user, takes an input pair and produces a set of *intermediate* key/value pairs. The MapReduce library groups together all intermediate values associated with the same intermediate key I and passes them to the reduce function

The reduce function, also written by the user, accepts an intermediate key I and a set of values for that key. It merges these values together to form a possibly smaller set of values.

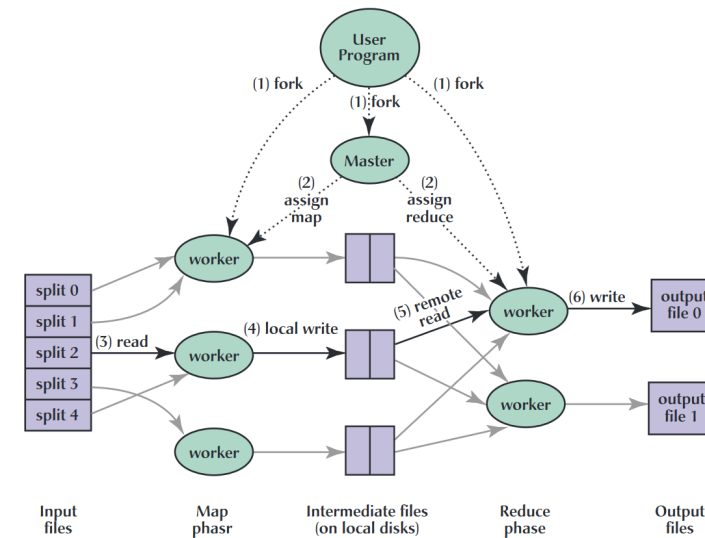


Fig. 1. Execution overview.

Shuffle

Implementation of `groupByKey` and `reduceByKey` :

- Map workers write intermediate data partitioned by hash to local disk
- Map workers notify master of output location
- Master keeps track of these locations
- Master tells reduce workers where to find their partitions
- Reduce workers pull their data over the network from those locations

Potentially M mappers times R reducers network messages.

demo/MapReduce.scala

Stream Fusion

Duncan Coutts, Roman Leshchinskiy, and Don Stewart. 2007. Stream fusion: from lists to streams to nothing at all.

Oleg Kiselyov, Aggelos Biboudis, Nick Palladinos, and Yannis Smaragdakis. 2017. Stream fusion, to completeness

Avoid materializing intermediate data structures in streaming pipelines.

```
trait Stream[A] { def next(): Option[A] }
```

Example pipeline:

```
xs.map(f).filter(p).map(g)
```

Fuse into a single object:

```
new Stream[C] {  
  def next() = xs.next() match {  
    case Some(x) =>  
      val y = f(x)  
      if (p(y)) Some(g(y))  
      else next()  
    case None => None  
  }  
}
```

Fusion in Spark

Narrow dependencies: Each partition of the parent RDD is used by at most one partition of the child RDD.

- Examples: `map`, `filter`, `flatMap`, `mapPartitions`
- No shuffle needed: each task processes one partition independently
- Can fuse operations together in a single stage, cheap!

Wide dependencies: Each partition of the parent RDD may be used by multiple partitions of the child RDD.

- Examples: `groupByKey`, `reduceByKey`, `join`, `repartition`
- Requires shuffle: data must be redistributed across the network
- Forces a stage boundary, expensive!

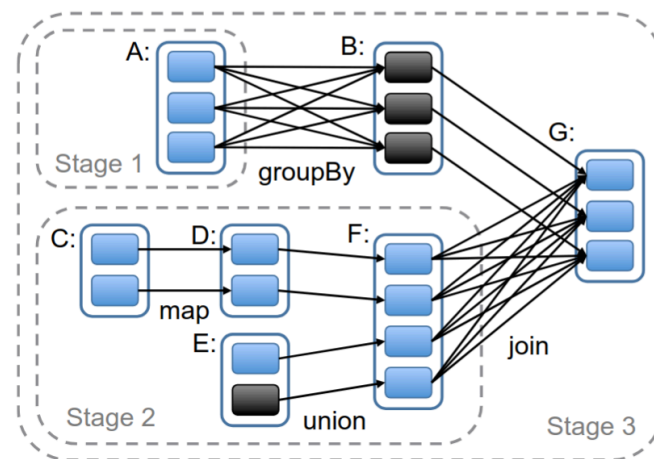


Figure 5: Example of how Spark computes job stages. Boxes with solid outlines are RDDs. Partitions are shaded rectangles, in black if they are already in memory. To run an action on RDD G, we build stages at wide dependencies and pipeline narrow transformations inside each stage. In this case, stage 1's output RDD is already in RAM, so we run stage 2 and then 3.

demo/Fusion.scala

Lineage

If no response is received from a worker in a certain amount of time, the master marks the worker as failed. Any map tasks completed by the worker are reset back to their initial idle state and therefore become eligible for scheduling on other workers. Similarly, any map task or reduce task in progress on a failed worker is also reset to idle and becomes eligible for rescheduling.

Jeffrey Dean and Sanjay Ghemawat. 2008. MapReduce: simplified data processing on large clusters.

[...] an RDD has enough information about how it was derived from other datasets (its *lineage*) to *compute* its partitions from data in stable storage. This is a powerful property: in essence, a program cannot reference an RDD that it cannot reconstruct after a failure.

Matei Zaharia, et al. 2012. Resilient distributed datasets: a fault-tolerant abstraction for in-memory cluster computing.

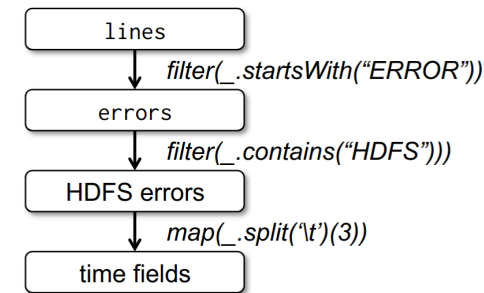


Figure 1: Lineage graph for the third query in our example. Boxes represent RDDs and arrows represent transformations.

Page Rank

Google's original algorithm for ranking search results.

Iterative algorithm simulating a random surfer.

When on a page, follows a random link, or jumps to a completely different page.

Calculates for each page the probability that the random surfer ends up on it.

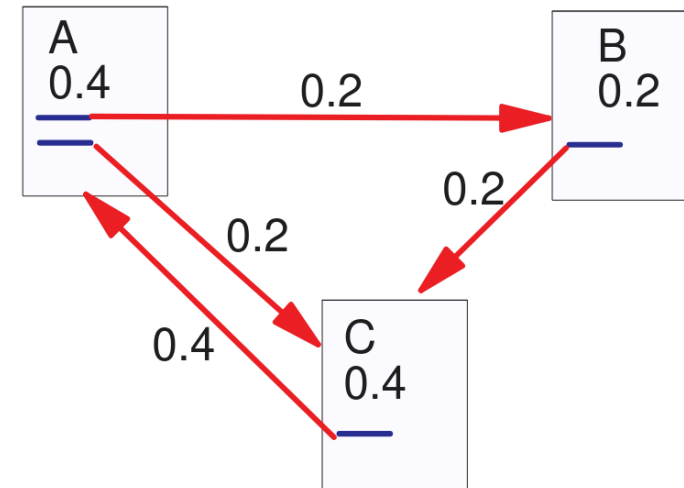


Figure 3: Simplified PageRank Calculation

Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. The PageRank citation ranking: bringing order to the web.

demo/PageRank.scala

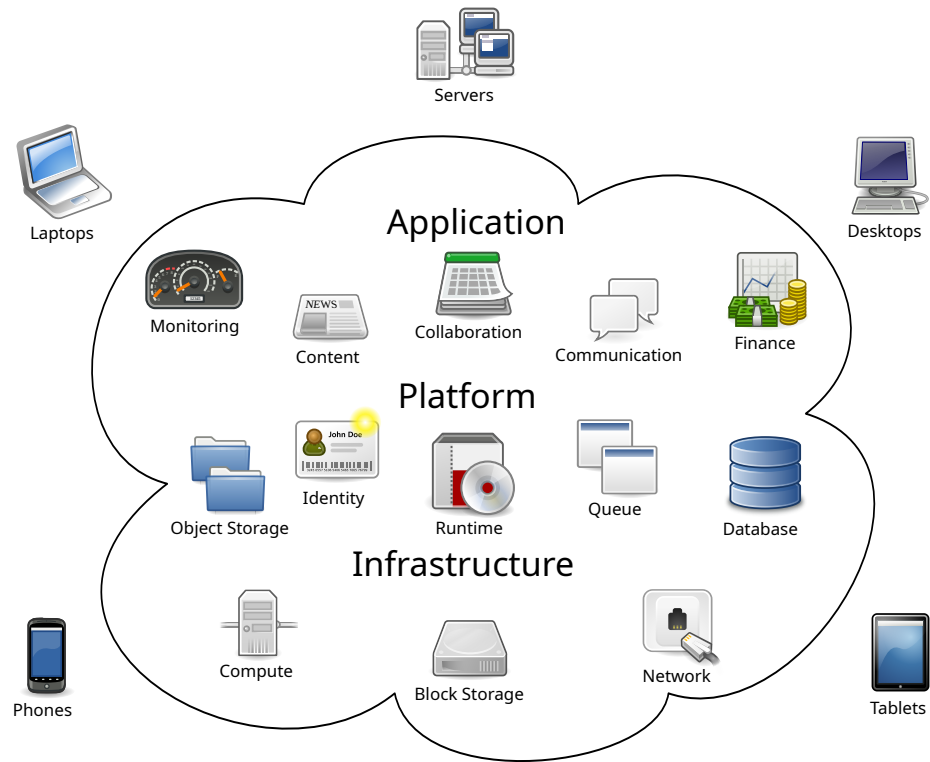
Summary and Outlook

Map Shuffle Reduce just works for many tasks.

Next week: Orchestration.

Orchestration

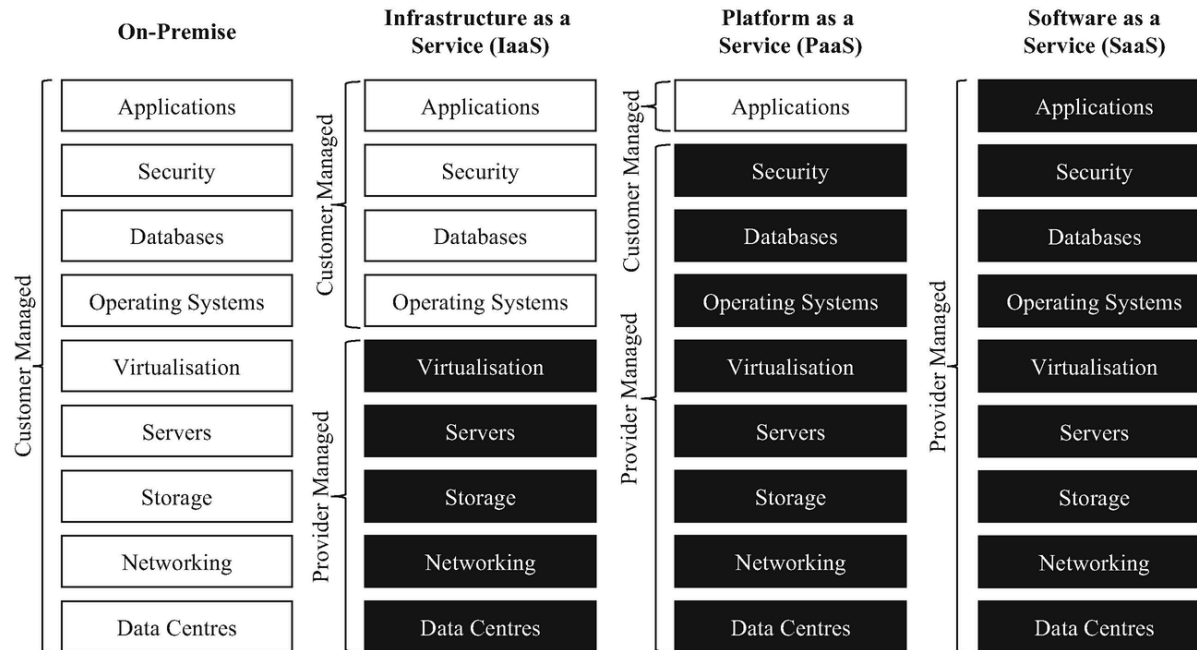
The Cloud



https://commons.wikimedia.org/wiki/File:Cloud_computing.svg

As A Service

Orchestration is necessary for On-Premise Installation, IaaS, and PaaS.



https://commons.wikimedia.org/wiki/File:Comparison_of_on-premise,_IaaS,_PaaS,_and_SaaS.png

Function as a Service (FaaS): provider runs individual functions in response to events.

Cloud Computing

Cloud Computing refers to both the applications delivered as services over the Internet and the hardware and systems software in the datacenters that provide those services.

[Developers with innovative ideas] need not be concerned about over-provisioning for a service whose popularity does not meet their predictions, thus wasting costly resources, or under-provisioning for one that becomes wildly popular, thus missing potential customers and revenue.

Moreover, companies with large batch-oriented tasks can get results as quickly as their programs can scale, since using 1000 servers for one hour costs no more than using one server for 1000 hour.

Michael Armbrust, et al. 2009. Above the Clouds: A Berkeley View of Cloud Computing.

When to use it

- Varying demand: traffic spikes, viral growth, seasonal patterns
- Batch processing: machine learning training
- Experimentation: spin up environments quickly and cheaply
- Startups: no upfront capital for infrastructure needed
- Global players: serve users across geographies

Job Scheduling

Stands in contrast to cloud computing.

Schedule long-running tasks onto a fixed number of machines in compute clusters and grids.

- limit resources used by individuals
- maximize overall resource utilization
- manages batch jobs in a queue

Examples: SLURM (on our university cluster), PBS, TORQUE, ...

This stands in contrast to managing a cloud of machines that continually offer services.

Virtualization

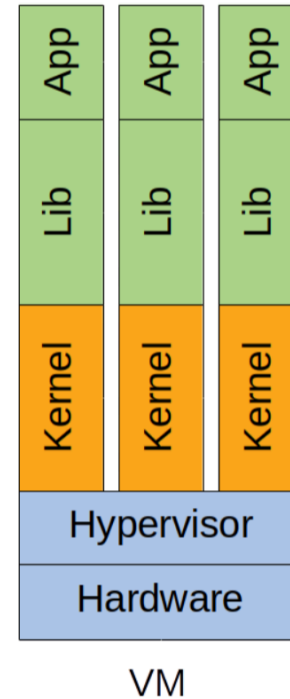
Hypervisor: runs directly on hardware and hosts multiple operating systems.

Advantage: isolation (separate kernels)

Disadvantage: heavy (size in GBs, boot time)

Examples: Xen, KVM, VMware, Hyper-V, ...

Enables Infrastructure as a Service.



Tom Goethals, et al. 2018. Unikernels vs Containers: An In-Depth Benchmarking Study in the Context of Microservice Applications.

Containerization

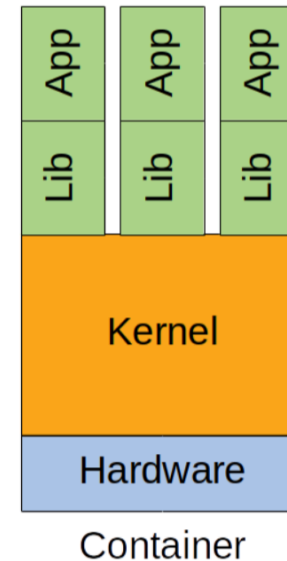
Containers: isolated processes that run on the same kernel but have their own filesystem and network, and limited resources.

Advantage: lightweight (size in MBs)

Disadvantage: weaker isolation (shared kernel)

Examples: Docker, LXC, containerd, CRI-O, ...

Enables Platform as a Service.



Tom Goethals, et al. 2018. Unikernels vs Containers: An In-Depth Benchmarking Study in the Context of Microservice Applications.

Unikernel

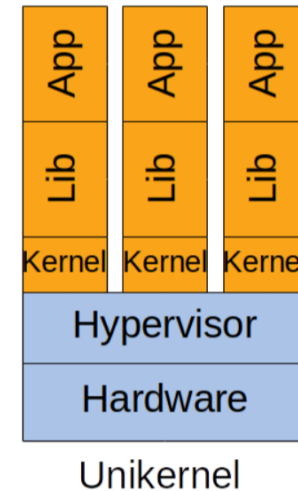
Unikernel: application compiled together with operating system components necessary to run on a hypervisor.

Advantage: very fast boot (time in ms)

Disadvantage: emerging technology

Examples: MirageOS, OSv, Unikraft, RumpRun, ...

Area of active research.



Tom Goethals, et al. 2018. Unikernels vs Containers: An In-Depth Benchmarking Study in the Context of Microservice Applications.

Kubernetes (K8s)

<https://kubernetes.io/>

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```

Released by Google in 2014.

Configuration Language

Infrastructure as code.

Imperative

Lists steps to execute.

```
docker run nginx
docker scale nginx --replicas=3
docker update nginx --image=nginx:1.16
```

Declarative

Describes desired state.

```
spec:
  replicas: 3
  image: nginx:1.16
```

Kubernetes Overview

Cluster

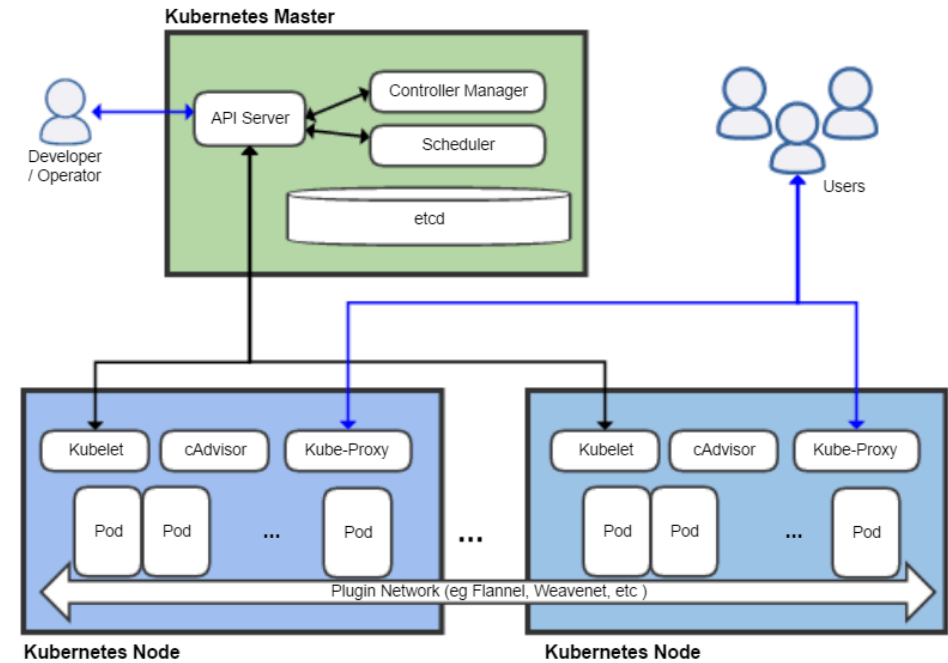
A set of machines managed by Kubernetes and treated as a single system.

Node

A single physical or virtual machine in the cluster.

Pod

Unit of deployment, contains containers that share a network and storage.



demo/basics/

Controllers

Desired state: specified by user and stored.

Observed state: continually reported and also stored.

Controllers reconcile observed state with desired state whenever there is a difference.

That's where the name Kubernetes comes from.

```
$ kubectl get pods -o name  
pod/nginx-66686b6766-lnbnc  
pod/nginx-66686b6766-lwj9  
pod/nginx-66686b6766-q7n8x
```

```
$ kubectl delete pod nginx-66686b6766-lnbnc
```

```
$ kubectl get pods -o name  
pod/nginx-66686b6766-bl9tj  
pod/nginx-66686b6766-lwj9  
pod/nginx-66686b6766-q7n8x
```


demo/delete_pod

Virtual Network

A software-defined network that provides the illusion of a single, unified network, independent of the underlying physical network.

Implemented with overlay networks: encapsulate packets to traverse underlying physical networks.

In Kubernetes:

Each Pod has a unique IP address independent of node structure.

Any Pod can reach any other Pod directly via its IP address.

Containers in the same Pod can communicate via localhost.

demo/network/

Service

A stable name for one or more locations of potentially changing servers.

Example: your phone contains a mapping from your friends' names to their numbers.

Common mechanisms:

- Configuration Files: hard-coded list of service locations.
- Environment Variables: inject service locations at startup.
- Domain Name System (DNS): map service name to one or more IP addresses.
- Service Registry: centralized database of service locations.

Service in Kubernetes

In Kubernetes, a Service is a method for exposing a network application that is running as one or more Pods in your cluster.

Every service automatically has:

- a virtual IP that never changes
- a DNS name `my-service.my-namespace.svc.cluster.local`
- a shorthand DNS name `my-service` from within the same namespace
- environment variables injected into Pods that start after the Service

A DNS server runs in the cluster and queries a distributed database `etcd` for the current Service to Pod mapping.

demo/service/

Load Balancing

How to distribute requests for a single service across multiple servers.

Common approaches:

- DNS round-robin: DNS server returns one of multiple addresses to client.
- Client-side: client has list of servers and picks one for example randomly.
- Proxy server: dedicated server between client and servers.

Layer 4 (transport layer): decide based on IP address and port.

Layer 7 (application layer): decide based on HTTP headers, path, cookies, etc.

Load Balancing in Kubernetes

Service Load Balancing

Splits requests to a service among multiple Pods.

Randomly chooses one of the Pods by default. Layer 4.

Gateway Load Balancing

Splits requests to a URL among multiple Services.

Programmable rules based on path, hostname, headers. Layer 7.

demo/balancing/

Summary and Outlook

Orchestration is a wide and deep topic.

Next week: Repetition.